

数据挖掘

杨柳

智能与计算学部

yangliuyl@tju.edu.cn



- 数据清理：如空缺值/噪声数据，离群点
- 数据集成：合并多个数据源
- 数据归约：
 - 属性归约
 - 数量归约
- 数据变换和数据离散化：
 - 规范化
 - 离散化

第 4 章 关联规则

4 关联规则

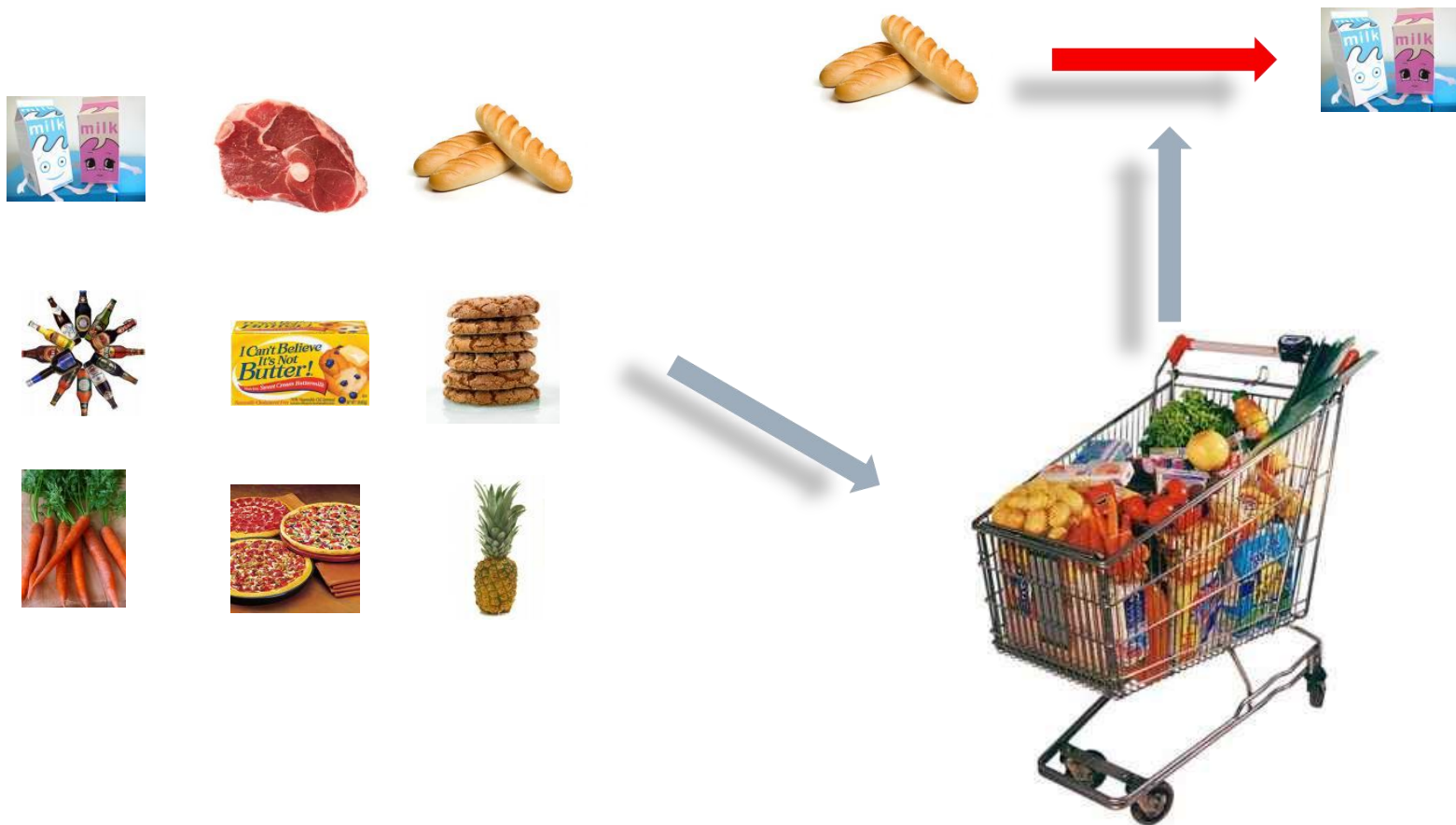


- 4.1 概述 
- 4.2 Apriori 算法
- 4.3 FP-growth 算法
- 4.4 推荐算法
- 4.5 小结

4.1 概述



- 关联规则是数据挖掘中最活跃的研究方法之一。
- 目的：发现交易数据库中不同商品之间的关联关系。



4.1 概述



购物篮分析

- 尿布与啤酒
- 电子产品
- 书



人气配件 鼠标 键盘 双肩包 显示器 移动硬盘 更多 ▾

商品名称	价格
Apple MacBook Air 13.3 英寸笔记本电脑 银色	
达尔优 (dare-u) S100 高端电竞游戏鼠标	¥139.00
多彩 (Delux) T9Plus 单手机机械键盘LOL游戏RGB	¥239.00
魅族 (MEIZU) 休闲旅行双肩包 灰色	¥199.00
戴尔 (DELL) SE2416H 23.8英寸窄边框带HDMI	¥899.00
耐泊尔 (NAIPOER) N-B1 高端2.5英寸移动硬盘	¥39.90

Customers Who Bought This Item Also Bought

商品名称	价格
 Handbook of Statistical Analysis and Data Mining... by John Elder IV ★★★★☆ (9) \$71.96	 Introduction to Data Mining by Pang-Ning Tan ★★★★★ (15) \$80.80
 Pattern Recognition and Machine Learning... by Christopher M. Bishop ★★★★☆ (47) \$58.03	 Machine Learning (Mcgraw-Hill International Edit) by Tom M. Mitchell ★★★★☆ (38) \$73.72
 Introduction to Machine Learning (Adaptive Comp... by Ethem Alpaydin ★★★★★ (8) \$43.79	

- 关联规则最早是由R.Agrawal(阿格拉沃尔)等人提出的。



- Agrawal R, Imieliński T, Swami A. [Mining association rules between sets of items in large databases](#). *Proceedings of the ACM International Conference on Management of Data, SIGMOD*, 1993, 207. (27526 year2026.9)
- Agrawal R, Srikant R. [Fast algorithms for mining association rules](#). *Proceedings of the 20th International Conference on Very Large Data Bases, VLDB*, 1994, 1215: 487-499. (31205 year 2026.9)

4 关联规则



- 4.1 概述
- 4.2 Apriori 算法 
- 4.3 FP-growth 算法
- 4.4 推荐算法
- 4.5 小结

引例

假定某超市销售的商品包括：bread、beer、cake、cream、milk和tea

交易号TID	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

□ 定义4.1 项目与项集

设 $I=\{i_1, i_2, \dots, i_m\}$ 是 m 个不同项目的集合，每个 $i_k(k=1, 2, \dots, m)$ 称为一个项目(Item)。

项目的集合称为项目集合(Itemset)，简称为项集。其元素个数称为项集的长度，长度为 k 的项集称为 k -项集(k -Itemset)。

交易号	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

基本概念中每个商品就是一个项目

项集 $I=\{\text{bread, beer, cake, cream, milk, tea}\}$ ，
长度为6，是6-项集

项集 $X=\{\text{milk, tea}\}$ ，2-项集

□ 定义4.2 交易

每笔交易 T (Transaction)是项集 I 上的一个非空子集, 即 $T \subseteq I$, 但通常 $T \subset I$ 。

对应每一个交易有一个唯一的标识——交易号, 记作TID

交易的全体构成了交易数据库 D , 或称交易记录集 D , 简称交易集 D 。

交易集 D 中包含交易的个数记为 $|D|$ 。

交易号	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

基本概念中包含10笔交易,
因此 $|D|=10$ 。

□ 定义4.3 项集的支持度

对于项集 X , $X \subseteq I$

绝对支持度（支持度计数） $\text{count}(X \subseteq T)$ 为交易集 D 中包含 X 的交易数量。

相对支持度项集 X 出现的概率，从而描述了 X 的重要性。

$$\text{support}(X) = \frac{\text{count}(X \subseteq T)}{|D|}$$

交易号	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

	支持度计数	支持度
$X_1 = \{\text{bread}\}$	7	0.7
$X_2 = \{\text{bread, milk}\}$	5	0.5
$X_3 = \{\text{bread, milk, tea}\}$	3	0.3

子集、超集

□ 定义4.4 项集的最小支持度与频繁集

发现关联规则要求项集必须满足的最小支持阈值，称为项集的最小支持度(Minimum Support)，记为 min_sup 。

从统计意义上讲，它表示用户关心的关联规则必须满足的最低重要性，只有满足最小支持度的项集才能产生关联规则。

支持度大于或等于 min_sup 的项集称为频繁项集，简称**频繁集**，反之则称为非频繁集。

对于 k -项集，若满足 min_sup ，称为 k -频繁集，记作 L_k 。

4.2.1 基本概念



交易号	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

如果设定 $\min_sup$ 为0.3,

$\{\text{bread, milk}\}$

支持度是0.5, 所以是2-频繁集。

$\{\text{cake, milk}\}$

支持度是0.2, 非频繁集。

□ 定义4.5 关联规则

关联规则(Association Rule)可以表示为一个蕴含式:

$R: X \Rightarrow Y$, 其中 $X \subset I, Y \subset I$, 并且 $X \cap Y = \emptyset$ 。

关联规则表示若项集 X 在某一交易中出现, 则会导致项集 Y 按照某一概率也会在同一交易中出现。

可以用两个标准来衡量: **支持度和置信度**。

□ 定义4.6 关联规则的支持度

对于关联规则 $R: X \Rightarrow Y$, 规则 R 的支持度(Support)是交易集中同时包含 X 和 Y 的交易数与所有交易数之比。

$$\text{support}(X \Rightarrow Y) = \frac{\text{count}(X \cup Y)}{|D|}$$

联合概率: $p(Y, X)$

4.2.1 基本概念



交易号	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

$$\text{support}(X \Rightarrow Y) = \frac{\text{count}(X \cup Y)}{|D|}$$

$X = \{\text{bread, milk}\}, Y = \{\text{tea}\}$

0.3

$Y = \{\text{bread, milk}\}, X = \{\text{tea}\}$

0.3

$X = \{\text{bread}\}, Y = \{\text{tea, cream}\}$

0.2

□ 定义4.7 关联规则的置信度

对于关联规则 $R: X \Rightarrow Y$ ，规则 R 的置信度(Confidence)是指包含 X 和 Y 的交易数与包含 X 的交易数之比：

$$\text{confidence } (X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)} = \frac{\text{count}(X \cup Y)}{\text{count}(X)}$$

置信度反映了若交易中包含 X ，则交易中同时出现 Y 的概率。

条件概率： $p(Y|X)$

一般来说，只有支持度和置信度较高的关联规则才是用户感兴趣的。

4.2.1 基本概念



交易号	顾客购买商品Items
T1	bread cream milk tea
T2	bread cream milk
T3	cake milk
T4	milk tea
T5	bread cake milk
T6	bread tea
T7	beer milk tea
T8	bread tea
T9	bread cream milk tea
T10	bread milk tea

$$\text{confidence } (X \Rightarrow Y) = \frac{\text{support}(X \cup Y)}{\text{support}(X)}$$

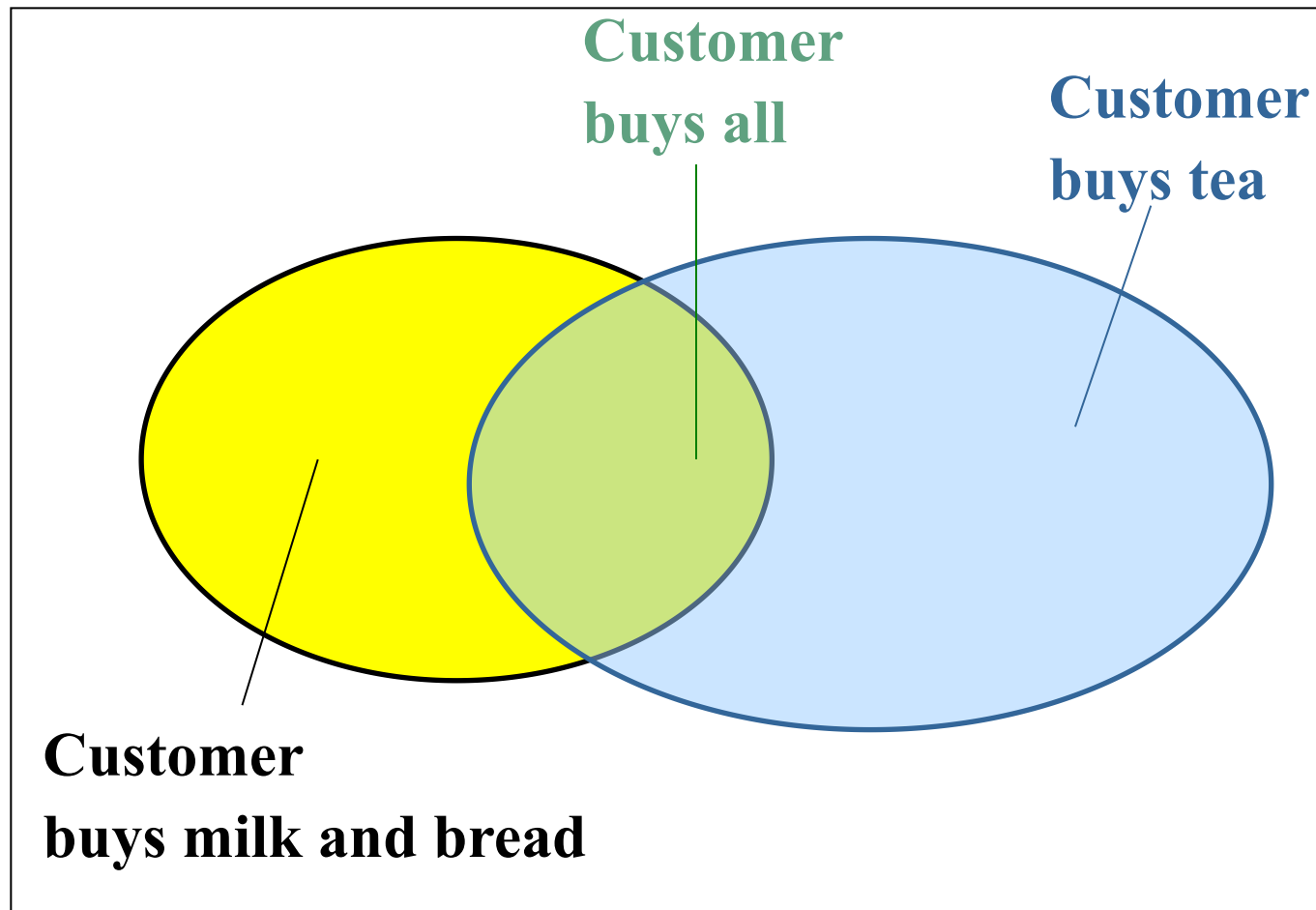
$$X = \{\text{bread, milk}\}, Y = \{\text{tea}\}$$

$$0.3/0.5$$

$$X = \{\text{tea}\}, Y = \{\text{bread, milk}\}$$

$$0.3/0.7$$

4.2.1 基本概念



□ 定义4.8 关联规则的最小支持度和最小置信度

关联规则的最小支持度：衡量频繁集的最小支持度 (Minimum Support)，记为 min_sup ，它是用于衡量规则需要满足的最低重要性。

关联规则的最小置信度 (Minimum Confidence) 记为 min_conf ，它表示关联规则需要满足的最低可靠性。

□ 定义4.9 强关联规则

如果规则 $X \Rightarrow Y$ 满足：

$\text{support}(X \Rightarrow Y) \geq \text{min_sup}$ 且 $\text{confidence}(X \Rightarrow Y) \geq \text{min_conf}$,

称关联规则 $X \Rightarrow Y$ 为强关联规则，否则为弱关联规则。

在挖掘关联规则时，产生的关联规则要经过 min_sup 和 min_conf 的衡量，筛选出来的强关联规则才能用于指导商家的决策。

表1：关联规则的简单例子

TID	网球拍	网 球	运动鞋	羽毛球
1	1	1	1	0
2	1	1	0	0
3	1	0	0	0
4	1	0	1	0
5	0	0	1	1
6	1	1	0	0

表1是顾客购买记录的数据库 D ，包含6个事务。
项集 $I=\{\text{网球拍}, \text{网球}, \text{运动鞋}, \text{羽毛球}\}$ 。

$\text{min_sup}=0.5$, $\text{min_conf}=0.5$

考虑关联规则：网球拍 $\Rightarrow$ 网球，

$$\text{支持度} = \frac{3}{6} = 0.5$$

$$\text{置信度} = \frac{3}{5} = 0.6$$

是强关联规则

- 大多数关联规则挖掘算法通常采用的一种策略是，将关联规则挖掘任务分解为如下两个主要的子任务：
 - 频繁项集产生 (Frequent Itemset Generation)
 - 其目标是发现满足最小支持度阈值的所有项集，这些项集称作频繁项集。
 - 规则的产生 (Rule Generation)
 - 其目标是从上一步发现的频繁项集中提取所有高支持度和置信度的规则，这些规则称作强规则 (strong rule)。找出支持度大于等于 min_sup 并且置信度大于等于 min_conf 的所有规则。

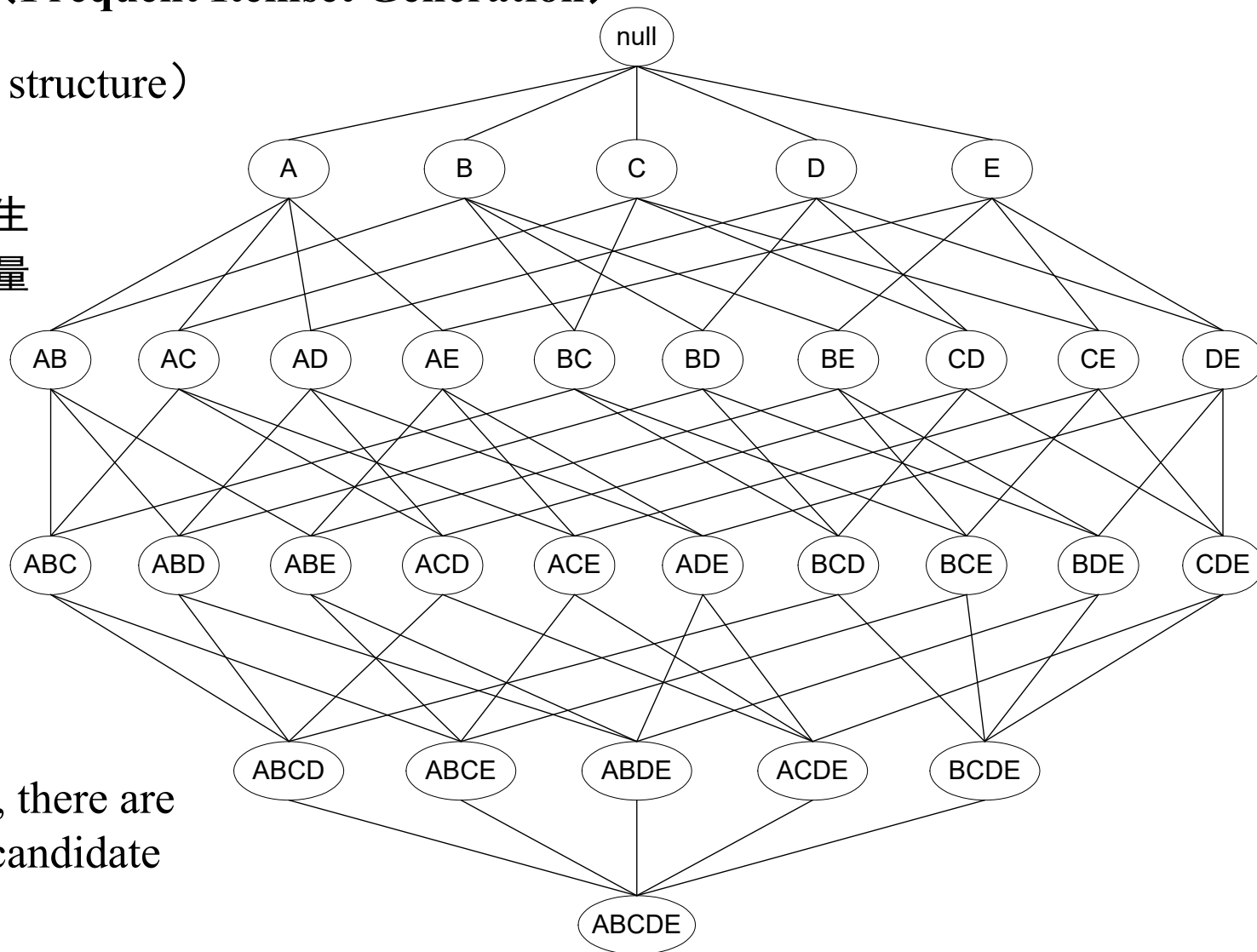
4.2.2 挖掘策略



频繁项集产生 (Frequent Itemset Generation)

格结构 (lattice structure)

由事务数据集产生的
频繁项集的数量
可能非常大。



Given d items, there are $2^d - 1$ possible candidate itemsets

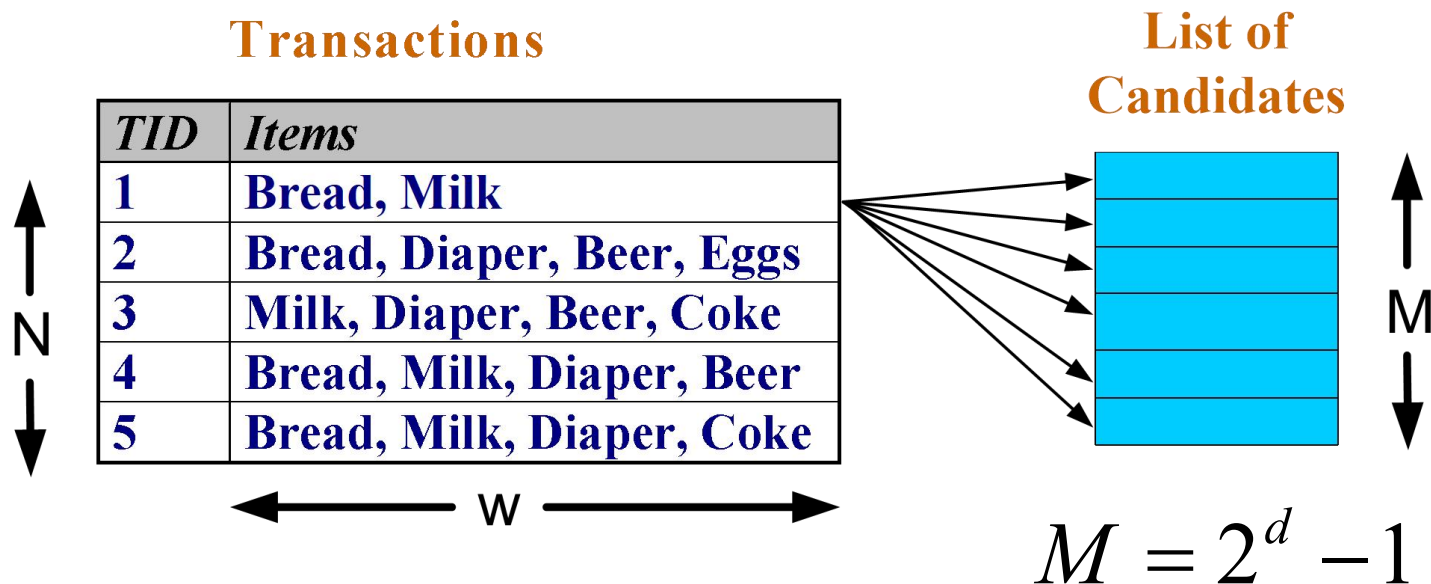
- 挖掘关联规则的一种原始方法是：Brute-force approach:
 - 计算每个可能规则的支持度和置信度
 - 这种方法计算代价过高，从数据集提取规则的数量达指数级

4.2.2 挖掘策略



□ Brute-force 方法:

- 把格结构中每个项集作为候选项集
- 将每个候选项集和每个事务进行比较，确定每个候选项集的支持度计数。



- 时间复杂度 $\sim O(NMW)$ ，这种方法的开销可能非常大。

降低产生频繁项集计算复杂度的方法

- 减少候选项集的数量
 - 先验(apriori)原理
- 减少比较的次数
 - 替代将每个候选项集与每个事务相匹配，可以使用更高级的数据结构，或存储候选项集或压缩数据集，来减少比较次数

4.2.3 Apriori算法



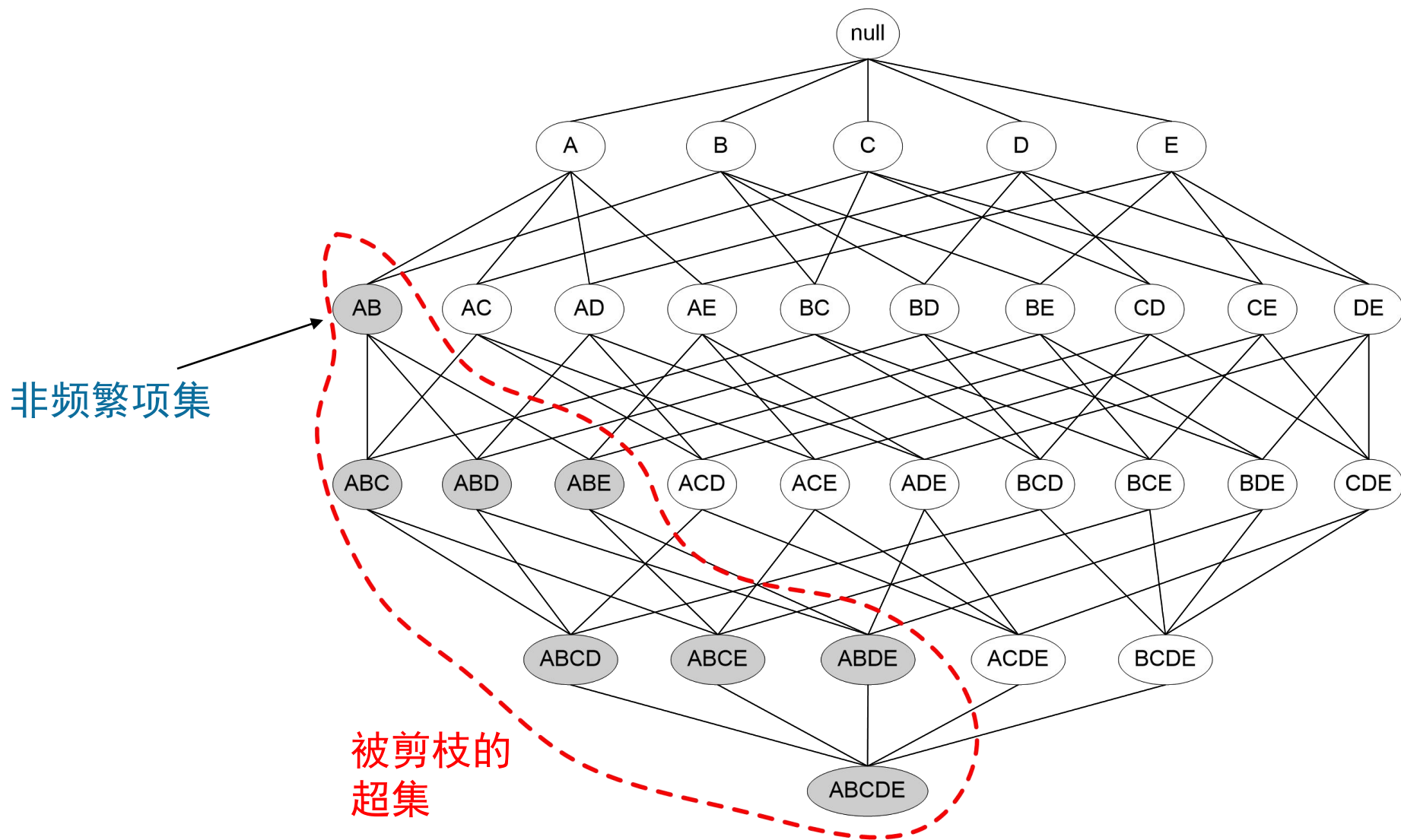
Apriori算法是一种最有影响的挖掘关联规则频繁项集的算法。算法名字是缘于算法使用了频繁项集的性质这一先验知识。

□ 先验原理：

- 如果一个项集是频繁的，则它的所有子集一定也是频繁的
- 相反，如果一个项集是非频繁的，则它的所有超集也一定是非频繁的

- 这种基于支持度度量修剪指数搜索空间的策略称为基于支持度的剪枝（support-based pruning）
- 这种剪枝策略依赖于支持度度量的一个关键性质，即一个项集的支持度决不会超过它的子集的支持度。这个性质也称为支持度度量的反单调性（anti-monotone）

4.2.3 Apriori算法



4.2.3 Apriori算法



在Apriori算法中，寻找频繁项集的基本思想是：算法需要对数据集进行多步处理。Apriori 使用了一种称作level-wise搜索的迭代方法。

- 找出频繁1项集，以 L_1 表示。 L_1 用来寻找 L_2 ，即频繁2-项集的集合。 L_2 用来寻找 L_3 ，以此类推，直至没有新的频繁 k -项集被发现。每个 L_k 都要求对数据库作一次完全扫描。
- 使用 L_{k-1} 找出 L_k ，由两步组成：
 - 连接步：将 L_{k-1} 与自身连接产生候选 k 项集的集合，标记为 C_k
 - 剪枝步：剪除不满足最小支持度的候选项集。

4.2.3 Apriori算法



- 下面以图例的方式说明该算法的运行过程： 假设有一个数据库 D ，其中有4个事务记录，预定最小支持度计数 $\text{min_sup}=2$ ，事务记录分别表示为：

TID	Items
T1	I1,I3,I4
T2	I2,I3,I5
T3	I1,I2,I3,I5
T4	I2,I5

4.2.3 Apriori算法



□ 扫描 D ，对每个候选项进行支持度计数得到

TID	Items
T1	I1,I3,I4
T2	I2,I3,I5
T3	I1,I2,I3,I5
T4	I2,I5

项集	支持度计数
{I1}	2
{I2}	3
{I3}	3
{I4}	1
{I5}	3

4.2.3 Apriori算法



- 比较候选项支持度计数与最小支持度 min_sup ，产生1项集

TID	Items
T1	I1,I3,I4
T2	I2,I3,I5
T3	I1,I2,I3,I5
T4	I2,I5

项集	支持度计数
{I1}	2
{I2}	3
{I3}	3
{I4}	1
{I5}	3

项集	支持度计数
{I1}	2
{I2}	3
{I3}	3
{I5}	3

4.2.3 Apriori算法



□ 产生候选项集

项集	支持度计数
{I1}	2
{I2}	3
{I3}	3
{I5}	3

项集
{I1,I2}
{I1,I3}
{I1,I5}
{I2,I3}
{I2,I5}
{I3,I5}

4.2.3 Apriori算法



□ 扫描 D ，对每个候选项集进行支持度计数：

TID	Items
T1	I1,I3,I4
T2	I2,I3,I5
T3	I1,I2,I3,I5
T4	I2,I5

项集
{I1,I2}
{I1,I3}
{I1,I5}
{I2,I3}
{I2,I5}
{I3,I5}

项集	支持度计数
{I1,I2}	1
{I1,I3}	2
{I1,I5}	1
{I2,I3}	2
{I2,I5}	3
{I3,I5}	2

4.2.3 Apriori算法



- 比较候选项支持度计数与最小支持度 min_sup ，产生2项集

项集	支持度计数
{I1,I2}	1
{I1,I3}	2
{I1,I5}	1
{I2,I3}	2
{I2,I5}	3
{I3,I5}	2

项集	支持度计数
{I1,I3}	2
{I2,I3}	2
{I2,I5}	3
{I3,I5}	2

4.2.3 Apriori算法



□ 产生候选项集

项集	支持度计数
{I1,I3}	2
{I2,I3}	2
{I2,I5}	3
{I3,I5}	2

项集
{I2,I3,I5}

4.2.3 Apriori算法



- 比较候选项支持度计数与最小支持度 min_sup ，产生3项集

TID	Items
T1	I1,I3,I4
T2	I2,I3,I5
T3	I1,I2,I3,I5
T4	I2,I5

项集	支持度计数
{I2,I3,I5}	2

- 算法终止

4.2.3 Apriori算法



□ 所有的频繁项集

项集	支持度计数
{I1}	2
{I2}	3
{I3}	3
{I5}	3

项集	支持度计数
{I1,I3}	2
{I2,I3}	2
{I2,I5}	3
{I3,I5}	2

项集	支持度计数
{I2,I3,I5}	2

4.2.3 Apriori算法



算法： (Apriori)利用层次循环发现频繁项集。

输入： 交易数据库 D ，最小支持阈值 min_sup ；

输出： L_i ， D 中的频繁项集；

处理流程：

- (1) $L_1 = \text{find_frequent_1_itemset}(D)$; //发现1-项集
- (2) for($k=2$; $L_{k-1} \neq \emptyset$; $k++$) {
- (3) $C_k = \text{apriori_gen}(L_{k-1}, \text{min_sup})$; //根据频繁($k-1$)-项集产生候选 k -项集
- (4) for each $t \in D$ { //扫描数据库，以确定每个候选项集的支持频度
- (5) $C_t = \text{subset}(C_k, t)$; //获得 t 所包含的候选项集
- (6) for each $c \in C_t$
- (7) $c.\text{count}++$;
- (8) }
- (9) $L_k = \{c \in C_k \mid c.\text{count} > \text{min_sup}\}$;
- (10) }
- (11) return $L = \bigcup_k L_k$;

4.2.3 Apriori算法



Apriori算法的频繁项集产生

<i>TID</i>	<i>Items</i>
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

最小支持度计数 = 3

4.2.3 Apriori算法



Apriori算法的频繁项集产生

Item	Count
Bread	4
Coke	2
Milk	4
Beer	3
Diaper	4
Eggs	1

Items (1-itemsets)



Itemset	Count
{Bread,Milk}	3
{Bread,Beer}	2
{Bread,Diaper}	3
{Milk,Beer}	2
{Milk,Diaper}	3
{Beer,Diaper}	3

Pairs (2-itemsets)



Triplets (3-itemsets)

Itemset	Count
{Bread,Milk,Diaper}	2

最小支持度计数 = 3

枚举所有项集将产生
 $C_6^1 + C_6^2 + C_6^3 = 41$ 个候选
而使用先验原理，将较少为
 $6 + 6 + 1 = 13$

- Apriori算法的频繁项集产生的部分有两个重要的特点：
 - 它是一个逐层算法。即从频繁1-项集到最长的频繁项集，它每次遍历项集格中的一层。
 - 它使用产生-测试策略来发现频繁项集。在每次迭代，新的候选项集由前一次迭代发现的频繁项集产生，然后对每个候选的支持度进行计数，并与最小支持度阈值进行比较。
- 该算法需要的迭代次数是 $k_{\max}$ ，其中 $k_{\max}$ 是频繁项集的最大长度

规则产生

- 忽略那些前件或后件为空的规则，每个频繁 k -项集能够产生多达 2^k-2 个关联规则
- 关联规则的提取：将一个项集 Y 划分成两个非空的子集 X 和 $Y-X$ ，使得 $X \Rightarrow Y-X$ 满足置信度阈值。
 - 如果 $\{A,B,C,D\}$ 是频繁项集，候选项集为：

$ABC \Rightarrow D,$ $ABD \Rightarrow C,$ $ACD \Rightarrow B,$ $BCD \Rightarrow A,$

$A \Rightarrow BCD,$ $B \Rightarrow ACD,$ $C \Rightarrow ABD,$ $D \Rightarrow ABC$

$AB \Rightarrow CD,$ $AC \Rightarrow BD,$ $AD \Rightarrow BC,$ $BC \Rightarrow AD,$

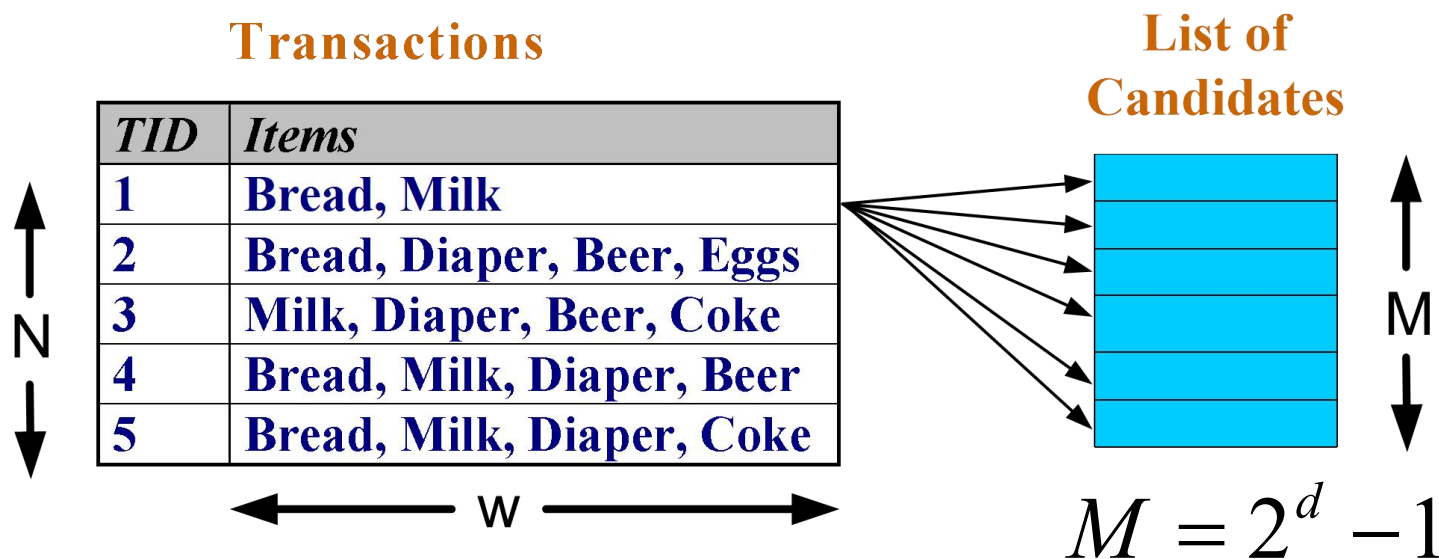
$BD \Rightarrow AC,$ $CD \Rightarrow AB$

- 这样的规则必然已经满足支持度阈值，因为它们是由频繁项₅

4.2.6 Apriori算法复杂度分析



- 把格结构中每个项集作为候选项集
- 将每个候选项集和每个事务进行比较，确定每个候选项集的支持度计数。



- 时间复杂度 $\sim O(NMW)$ ，这种方法的开销可能非常大。

4.2.6 Apriori算法复杂性分析



□ 支持度阈值

- 降低支持度阈值通常将导致更多的项集是频繁的，计算复杂度增加
- 随着支持度阈值的降低，频繁项集的最大长度将增加，导致算法需要扫描数据集的次数也将增多

□ 项数

- 随着项数的增加，需要更多的空间来存储项的支持度计数。如果频繁项集的数目也随着数据项数增加而增长，则由于算法产生的候选项集更多，计算量和I/O开销将增加

□ 事务数

- 由于Apriori算法反复扫描数据集，因此它的运行时间随着事务数增加而增加

□ 事务的平均宽度

- 频繁项集的最大长度随事务平均宽度增加而增加
- 随着事务宽度的增加，事务中将包含更多的项集，这将增加支持度计数时的遍历次数

4.2.7 Apriori算法优缺点



□ Apriori算法的优点：

- 简单、易理解、数据要求低

□ Apriori算法的缺点：

- 在每一步产生候选项目集时循环产生的组合过多，没有排除不应该参与组合的元素；
- 每次计算项集的支持度时，都对数据库 D 中的全部记录进行了一遍扫描比较，如果是一个大型的数据库的话，这种扫描比较会大大增加计算机系统的I/O开销。而这种代价是随着数据库的记录的增加呈现出几何级数的增加。

4.2.7 Apriori算法优缺点



Apriori算法的瓶颈

- 模式：候选项集——生成——测试
- 主要开销
 - 生成候选项集是高代价的
 - 例如数据库中包含100个项目 $i_1i_2...i_{100}$
候选项集的数量：
$$C_{100}^1 + C_{100}^2 + ... + C_{100}^{100} = 2^{100} - 1$$
 - 多次重复扫描数据库
- 如何提高生成频繁项集的效率？

- 4.1 概述
- 4.2 Apriori 算法
- 4.3 FP-growth 算法 
- 4.4 推荐算法
- 4.5 小结

- 针对Apriori算法的性能瓶颈问题——需要产生大量候选项集和需要重复地扫描数据库。
- 2000年Jiawei Han等人提出了基于FP-树生成频繁项集的FP-growth算法。该算法不同于Apriori算法的“产生-测试”范型，是不生成候选项集的关联规则方法。
- FP-树是一种输入数据的压缩表示，它通过逐个读入事务，并把每个事务映射到FP-树中的一条路径来构造。

Han J, Pei J, Yin Y. Mining frequent patterns without candidate generation[J]. ACM sigmod record, 2000, 29(2): 1-12. (引用次数9724, 2021.9)

Han J, Pei J, Yin Y, et al. Mining frequent patterns without candidate generation: A frequent-pattern tree approach[J]. Data mining and knowledge discovery, 2004, 8(1): 53-87. (引用次数3215, 2021.9)

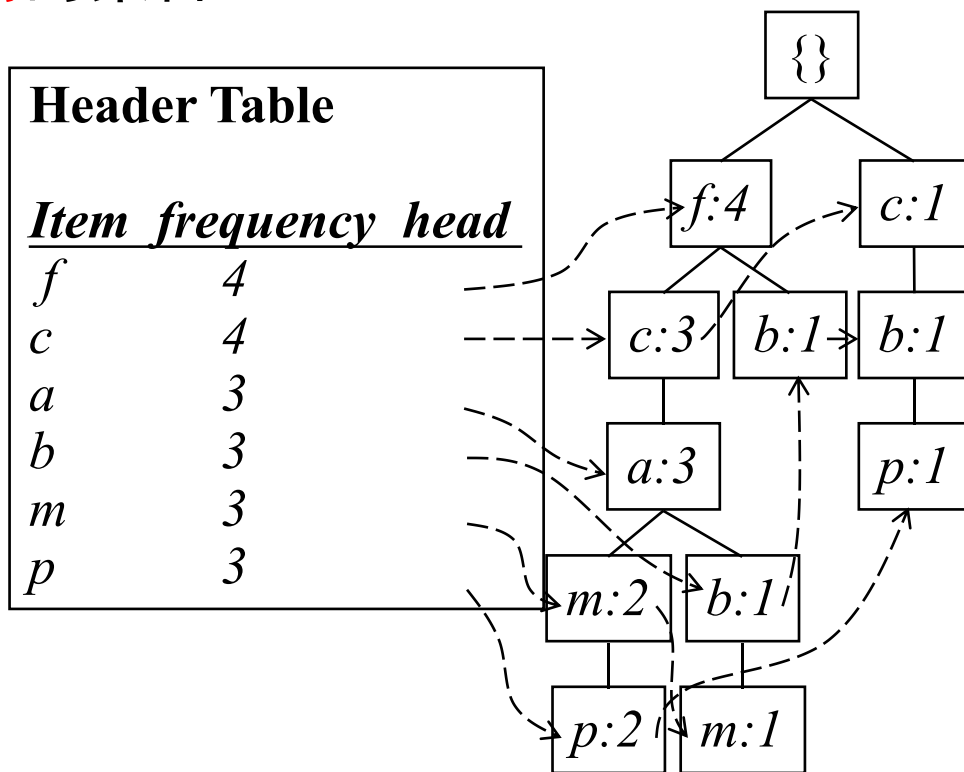
4.3 FP-growth算法



- FP-growth是一种以自底向上方式探索树，由FP-树产生频繁项集的算法，递归地频繁增长模式，由两个阶段组成：
 - 第一阶段为建立FP-树，即将数据库中的事务构造成一棵FP-树；
 - 第二阶段为挖掘FP-树，即针对FP-树挖掘频繁模式和关联规则。
- 该算法只进行2次数据库扫描且它不使用候选集，直接压缩数据库成一个频繁模式树，最后通过这棵树生成关联规则。研究表明它比Apriori算法大约快一个数量级。
 - 第一次扫描数据库，得到1维频繁项集；
 - 第二次扫描数据库，利用1维频繁项集过滤数据库中的非频繁项，同时生成FP-树。

□ 定义4.10 FP-树

频繁模式树FP-树是一个树形结构。包括一个**频繁项组成的头表**，一个标记为“null”的根节点，它的子节点为一个**项前缀子树**的集合。



□ 定义4.11 频繁项头表

频繁项头表(Head Table)的每个表项通常由两个域组成：项目名称 item-name 和指针 node_link。node_link 指向FP-树中具有与该表项相同 item-name 的第一个节点。有时也包含每个项目的支持度计数。

□ 定义4.12 项前缀子树

每个项前缀子树(Item Prefix Subtree)的节点有三个域：item-name, count, node_link。item-name 记录了该节点所代表的项的名字。count 记录了所在路径代表的交易中包含此节点项目的交易个数。node_link 指向下一个具有同样的 item-name 域的节点，要是没有这样一个节点，就为 null。

第一阶段：根据一个输入交易记录集建立一棵FP-树。

- 输入：交易记录集D，最小支持度 sup_min 。
- 输出：FP-树。
 - 扫描数据库D一遍，得到频繁项的集合F和每个频繁项的支持度。把F按支持度递减排序，结果记为L。
 - 创建FP-树的根节点，记为T，并且标记为“null”。
 - 第二次扫描数据库D。每个事务中的项按L中的次序排列，并对每个事务创建分枝。一般地，当为一个事务考虑增加分枝时，沿共同前缀上的每个结点计数增加1，为前缀之后的创建结点和链接。

4.3 FP-growth算法



构造FP-树

$min_sup = 2$

TID	Items
1	{B,A}
2	{B,D,C}
3	{E,C,D,A}
4	{A,E,D}
5	{C,A,B}
6	{D,B,A,C}
7	{A}
8	{B,A,C}
9	{B,A,D}
10	{B,E,C}



Items	Frequency
A	8
B	7
C	6
D	5
E	3



TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

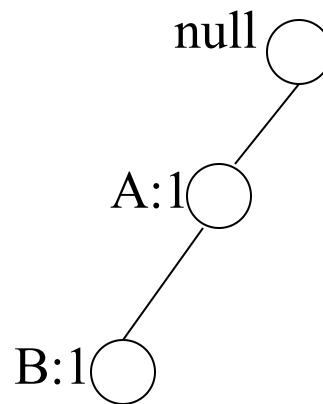
4.3 FP-growth算法



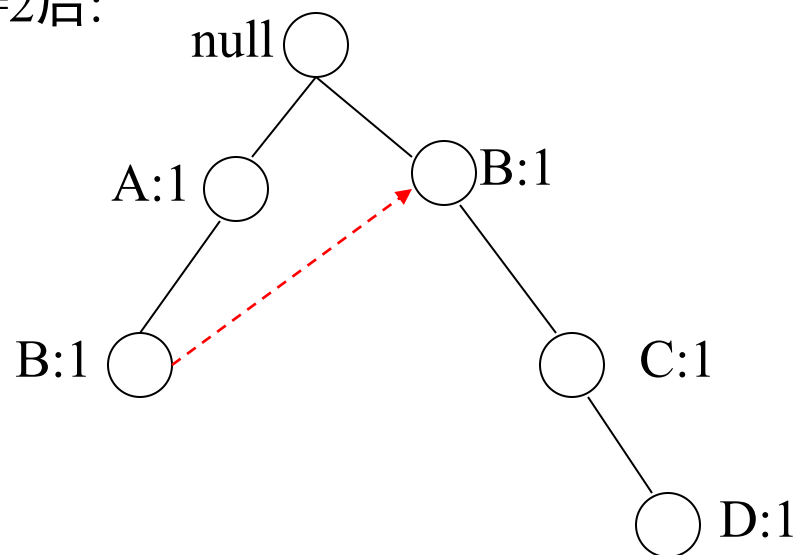
构造FP-树

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

读入事务 TID=1后:



读入事务 TID=2后:

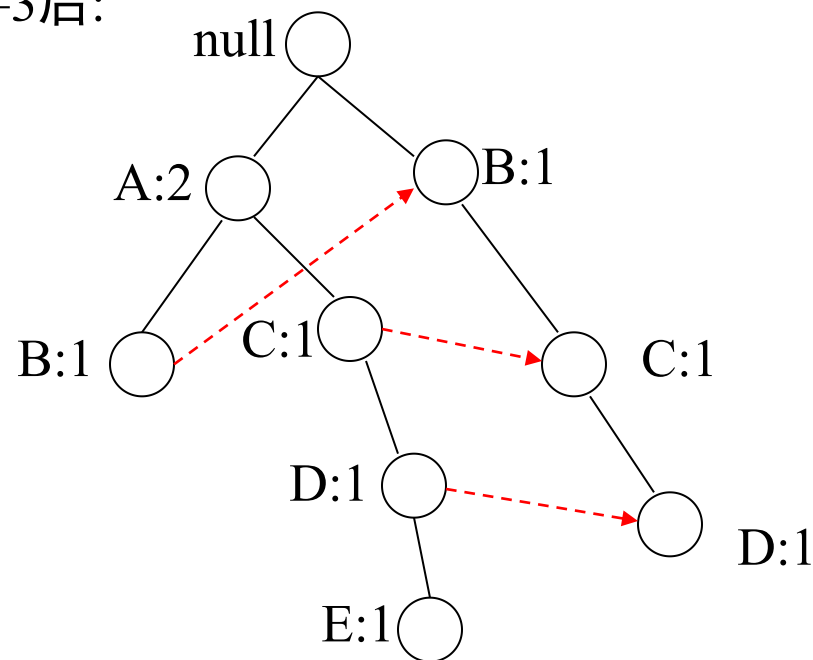


- 第三个事务{A, C, D, E}与第一个事务共享一个共同的前缀项A，所以第三个事务的路径 $\text{null} \rightarrow A \rightarrow C \rightarrow D \rightarrow E$ 与第一个事务的路径 $\text{null} \rightarrow A \rightarrow B$ 部分重叠。因为它们的路径重叠，所以结点A的频度计数增加为2.
- 继续该过程，直到每个事务都映射到FP-树的一条路径。

构造FP-树

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}

读入事务 TID=3后:

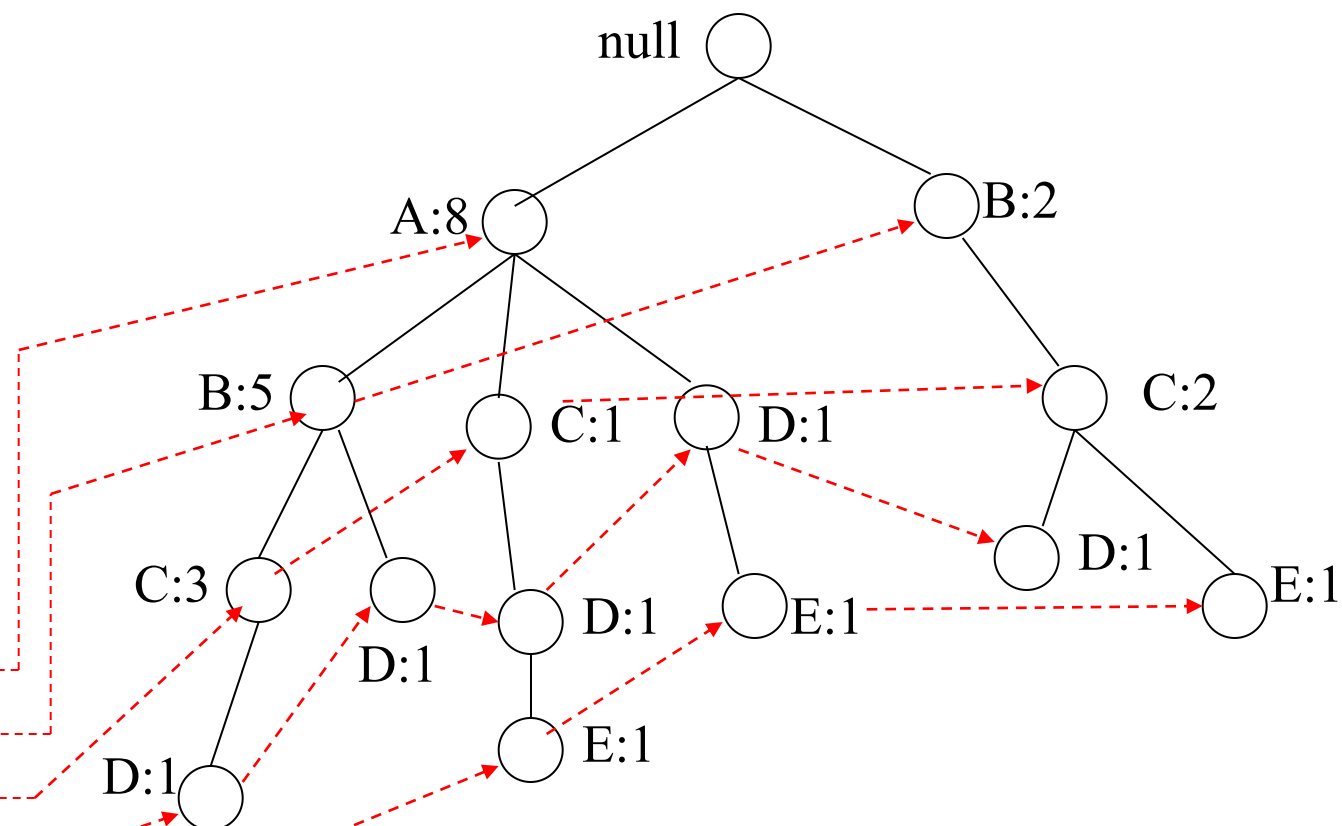


4.3 FP-growth算法



构造FP-树

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,C,D,E}
4	{A,D,E}
5	{A,B,C}
6	{A,B,C,D}
7	{A}
8	{A,B,C}
9	{A,B,D}
10	{B,C,E}



Header Table
Item frequency head

A	8
B	7
C	6
D	5
E	3

4.3 FP-growth算法



<i>TID</i>	<i>Items bought</i>
100	{ <i>f, a, c, d, g, i, m, p</i> }
200	{ <i>a, b, c, f, l, m, o</i> }
300	{ <i>b, f, h, j, o, w</i> }
400	{ <i>b, c, k, s, p</i> }
500	{ <i>a, f, c, e, l, p, m, n</i> }

$\text{min_sup} = 3$

4.3 FP-growth算法



由事务数据库构建FP-树

<i>TID</i>	<i>Items bought</i>	<i>(ordered) frequent items</i>
100	{ <i>f, a, c, d, g, i, m, p</i> }	{ <i>f, c, a, m, p</i> }
200	{ <i>a, b, c, f, l, m, o</i> }	{ <i>f, c, a, b, m</i> }
300	{ <i>b, f, h, j, o, w</i> }	{ <i>f, b</i> }
400	{ <i>b, c, k, s, p</i> }	{ <i>c, b, p</i> }
500	{ <i>a, f, c, e, l, p, m, n</i> }	{ <i>f, c, a, m, p</i> }

$min_sup = 3$

1. 扫描DB一次找到频繁1项
(单一项模式)
2. 按支持度降序对频繁项排序为 F-list
3. 再次扫描DB, 构建FP-树

Header Table

<i>Item frequency head</i>	
<i>f</i>	4
<i>c</i>	4
<i>a</i>	3
<i>b</i>	3
<i>m</i>	3
<i>p</i>	3

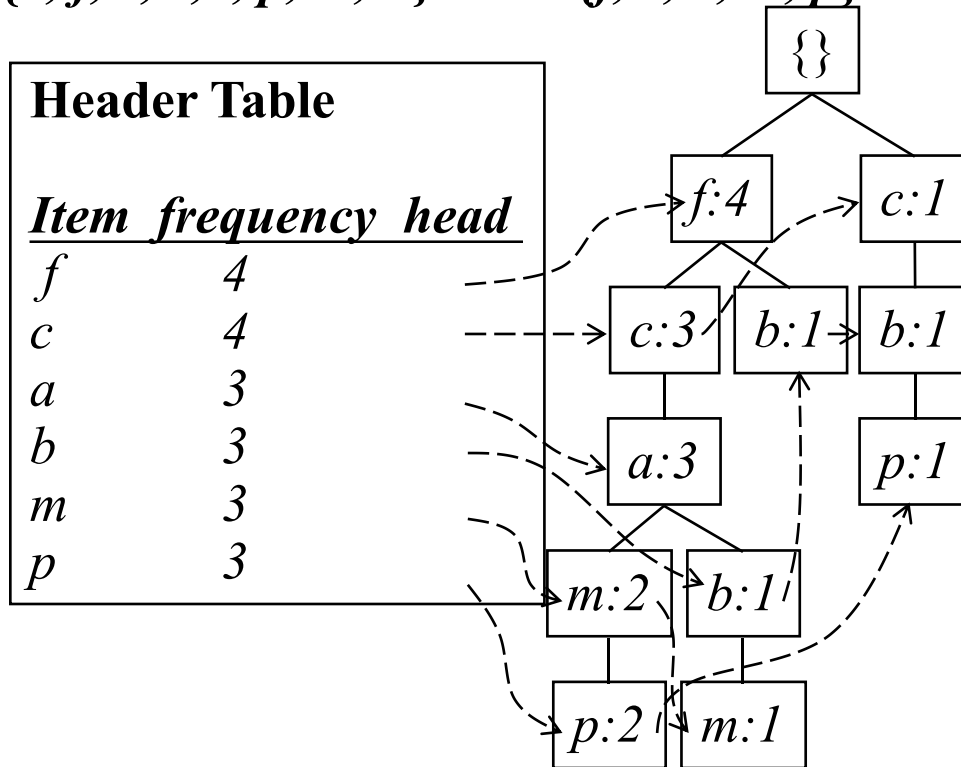
4.3 FP-growth算法



由事务数据库构建FP-树

<i>TID</i>	<i>Items bought</i>	<i>(ordered) frequent items</i>
100	{ <i>f, a, c, d, g, i, m, p</i> }	{ <i>f, c, a, m, p</i> }
200	{ <i>a, b, c, f, l, m, o</i> }	{ <i>f, c, a, b, m</i> }
300	{ <i>b, f, h, j, o, w</i> }	{ <i>f, b</i> }
400	{ <i>b, c, k, s, p</i> }	{ <i>c, b, p</i> }
500	{ <i>a, f, c, e, l, p, m, n</i> }	{ <i>f, c, a, m, p</i> }

$min_sup = 3$



F-list=*f-c-a-b-m-p*

构造FP-树

- 通常，FP-树的大小比未压缩的数据小，因为关联规则的数据中事务常常共享一些共同项。如果共同项较少，FP-树对存储空间的压缩效果将不明显。
- FP-树的大小也依赖于项如何排序。一般按照支持度计数递减序可以导致较小的FP-树。
- FP-树还包含一个连接具有相同项的结点的指针列表。这些指针有助于方便快捷地访问树中的项。

第二阶段为挖掘FP-树

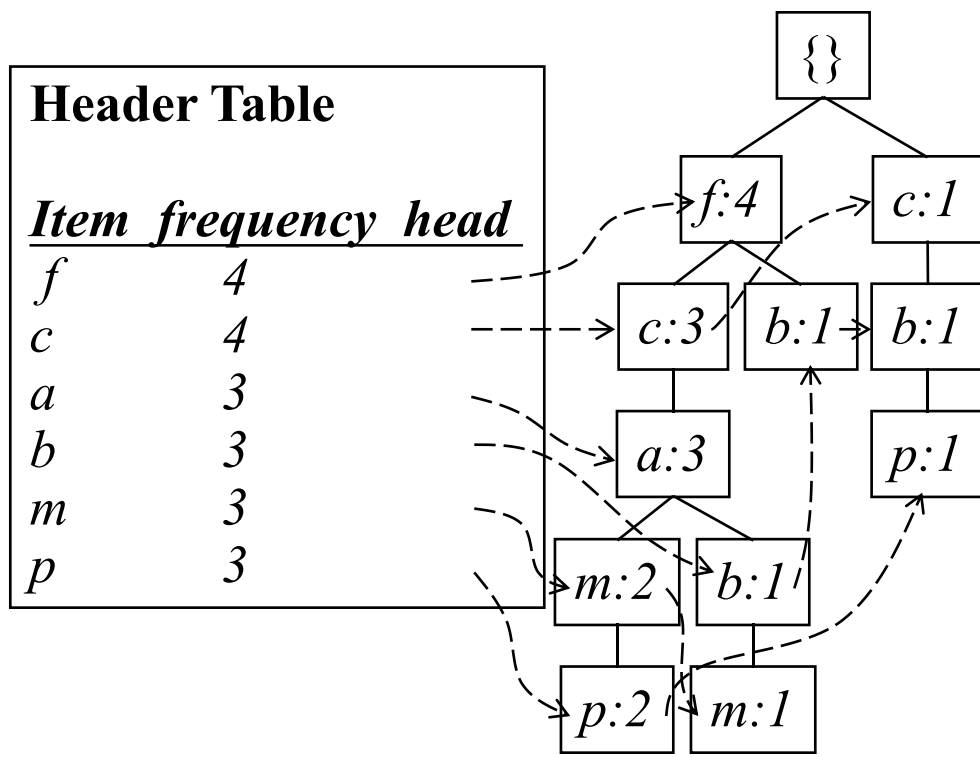
- 输入：FP-树。
- 输出：针对FP-树挖掘频繁模式和关联规则。
- 由于每一个事务都映射到FP-树中的一条路径，因而通过仅考察包含特定结点（例如 i ）的途径，就可以发现以 i 结尾的频繁项集。使用与结点 i 相关联的指针，可以快速访问这些路径。

□ 定义4.13 条件模式基

一个“子数据库”，由FP-树中与该后缀模式一起出现的前缀路径集组成。

□ 定义4.14 条件FP-树

由条件模式基构造的FP-子树



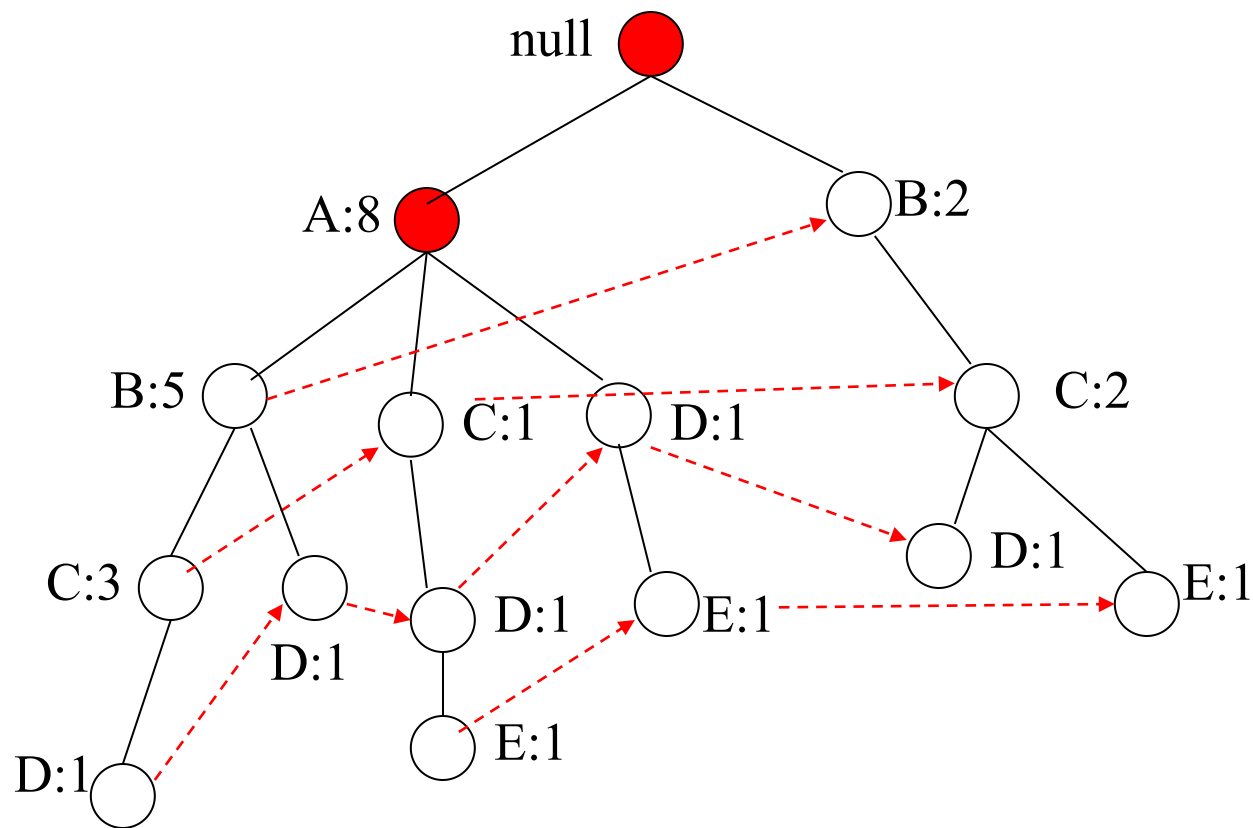
项	条件模式基
<i>c</i>	<i>f:3</i>
<i>a</i>	<i>fc:3</i>
<i>b</i>	<i>fca:1, f:1, c:1</i>
<i>m</i>	<i>fca:2, fcab:1</i>
<i>p</i>	<i>fcam:2, cb:1</i>

4.3 FP-growth算法



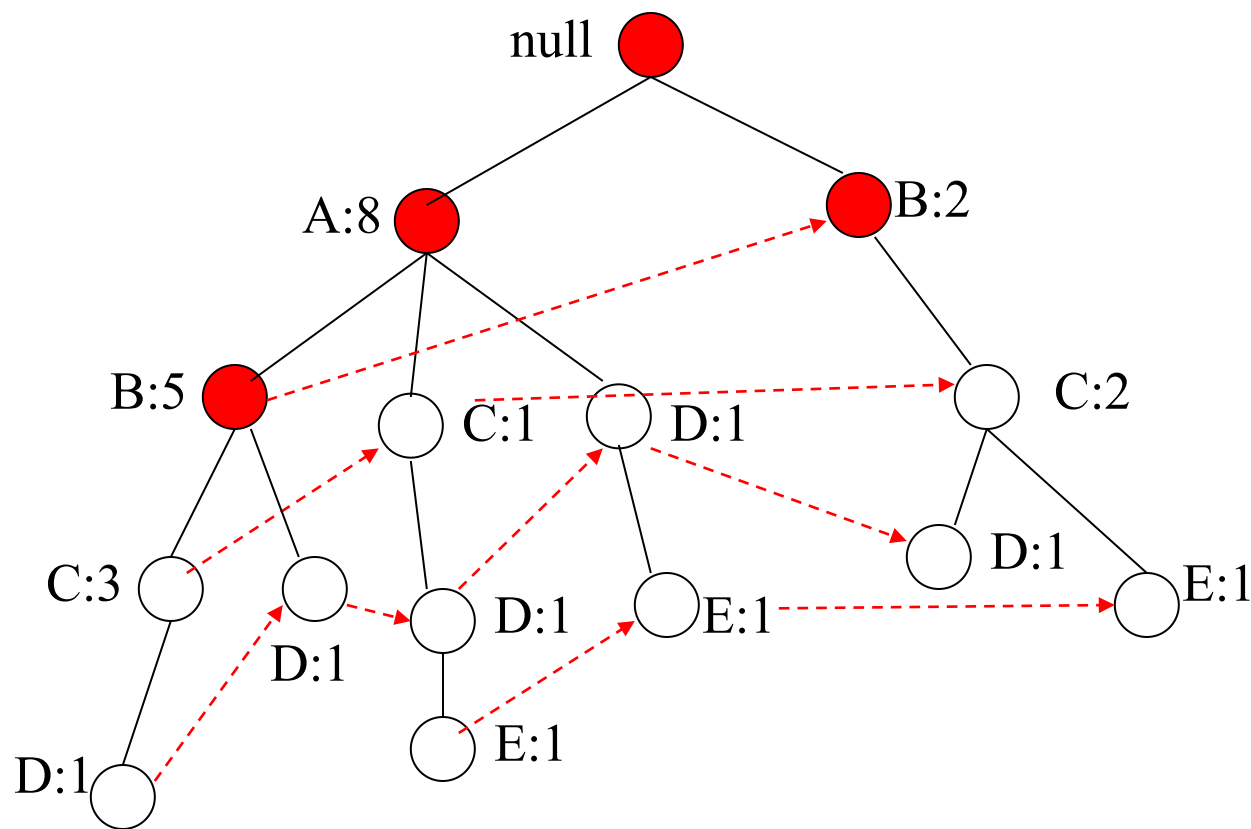
- **Step1:** 读取数据库，构造频繁1项集及FP-树
- **Step2:** 遍历FP-树的头表，对于每个频繁项 i ，累积项 i 的所有前缀路径形成 i 的条件模式基
- **Step3:** 对条件模式基上每一条路径的节点更新计数为 i 的计数，根据条件模式基构造条件FP-树
- **Step4:** 从条件FP-树中找到所有的路径，对该路径上的节点找出所有组合方式，然后合并计数
- **Step5:** 将Step4中的频繁项集与 i 合并，得到包含 i 的频繁项集
- **Step2-5** 循环，直到遍历头表中的所有项

4.3 FP-growth算法



$\{A\}$

4.3 FP-growth算法

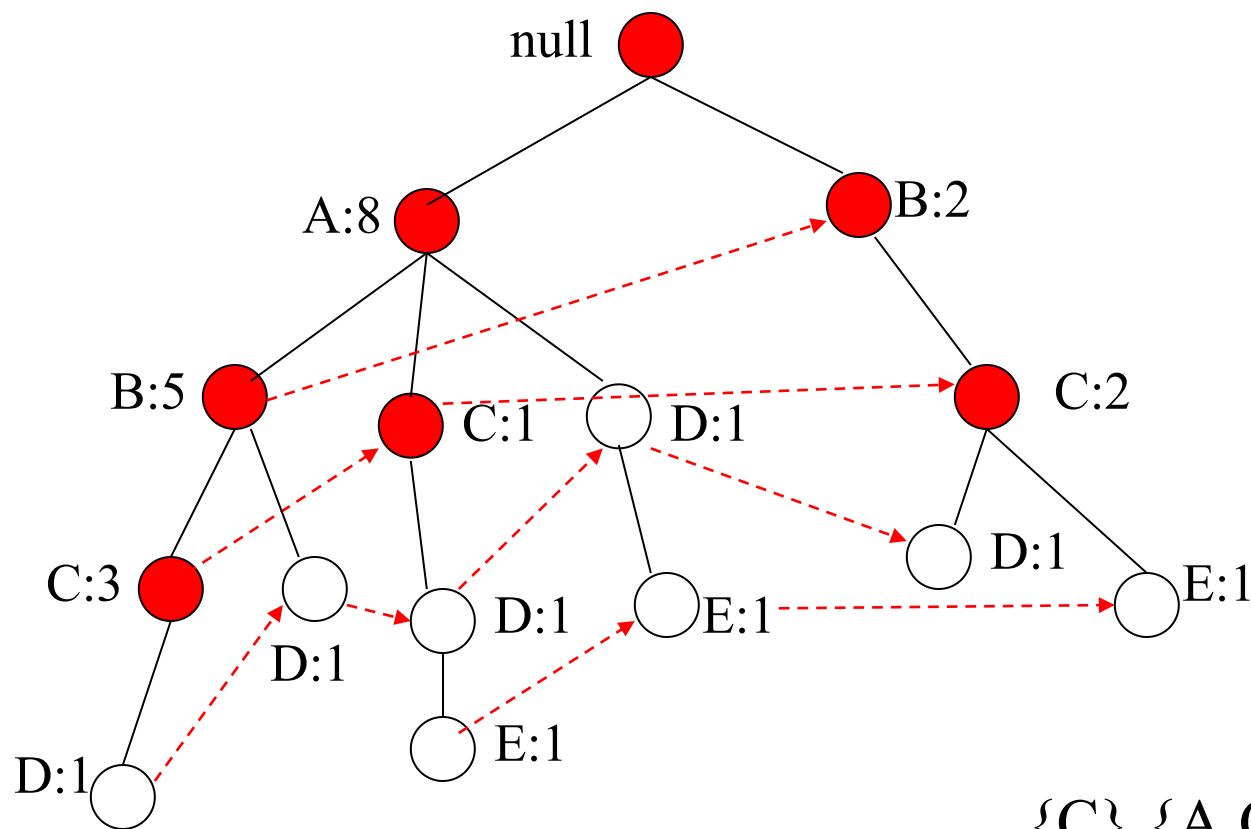


项 条件模式基

B A:5

$\{B\}, \{A,B\}$

4.3 FP-growth算法

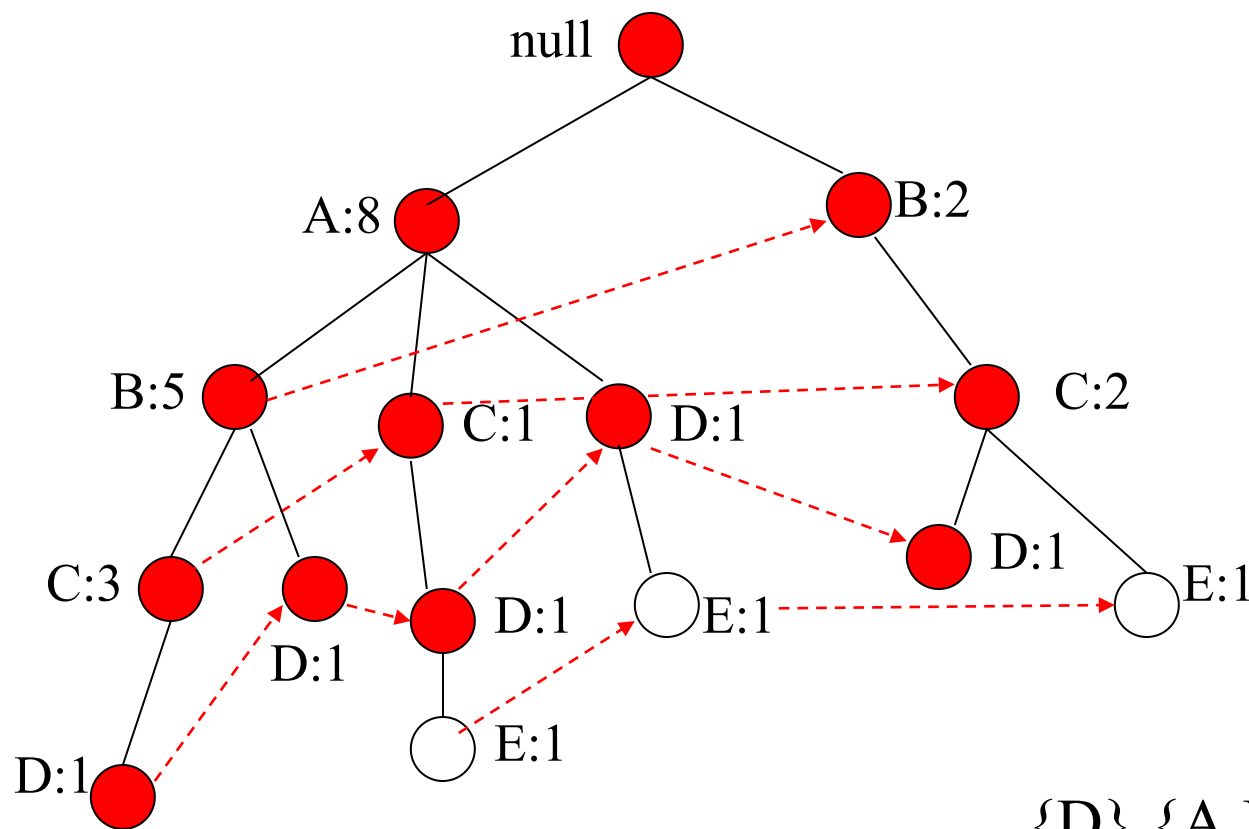


项	条件模式基
C	AB:3,A:1,B:2

合并计数 A:4,B:5,AB:3

$\{C\}, \{A,C\}, \{B,C\}, \{A,B,C\}$

4.3 FP-growth算法

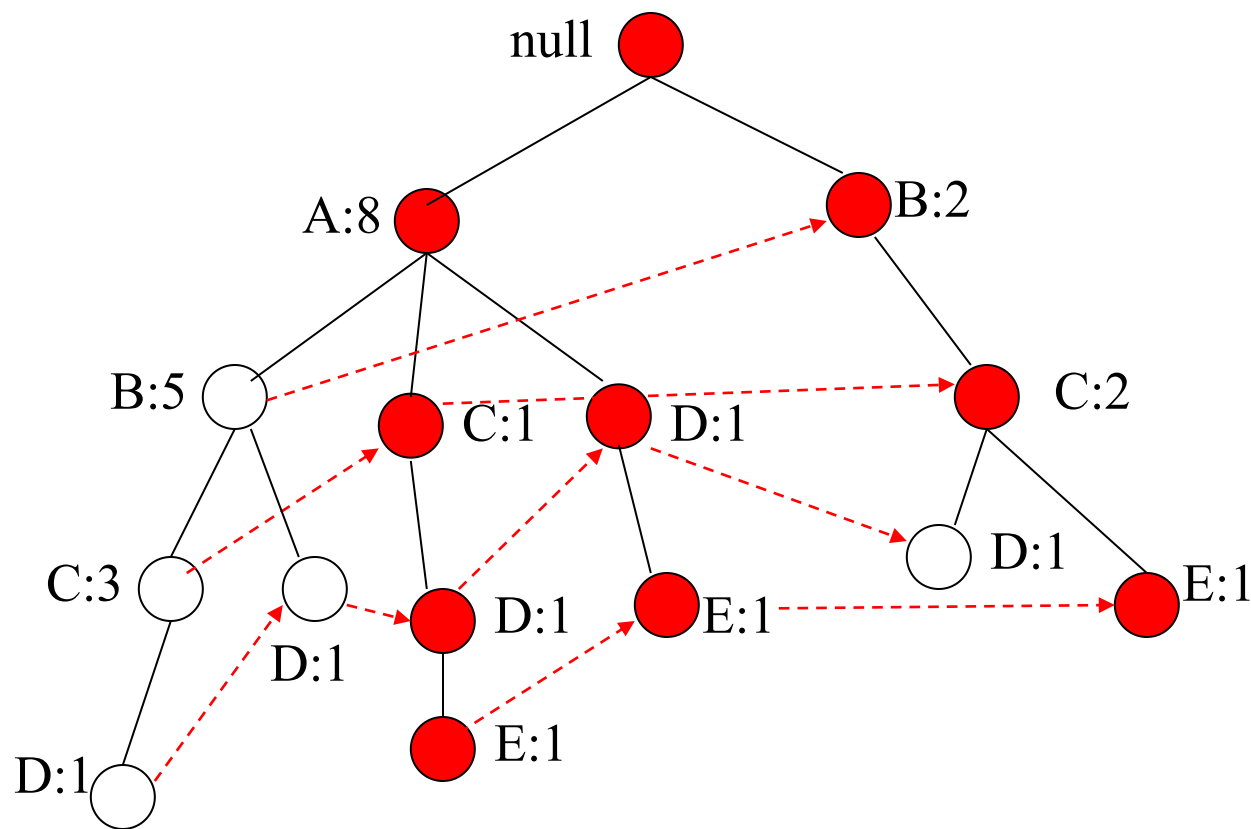


项	条件模式基
D	ABC:1, AB:1, AC:1,A:1,BC:1

合并计数 A:4,B:3,C:3
AB:2,AC:2,BC:2

$\{D\}, \{A,D\}, \{B,D\}, \{C,D\},$
 $\{A,B,D\}, \{A,C,D\}, \{B,C,D\}$

4.3 FP-growth算法



项 条件模式基

E ACD:1,AD:1

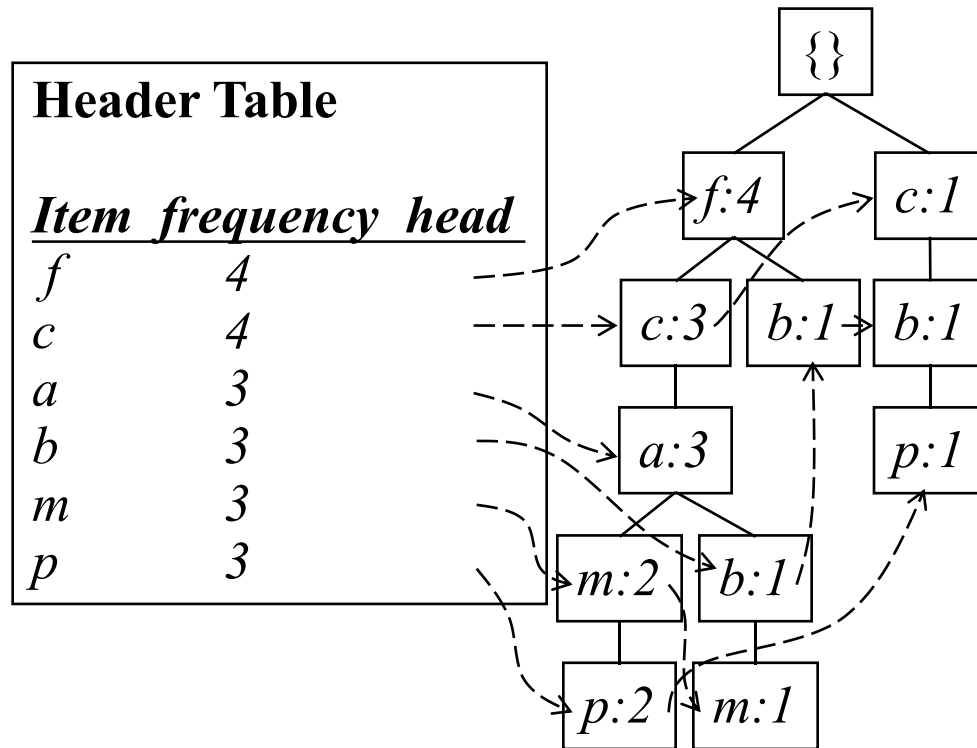
BC:1

合并计数 A:2,C:2,D:2

AD:2

$\{E\}, \{A,E\}, \{C,E\}, \{D,E\}, \{A,D,E\}$

4.3 FP-growth算法



项	条件模式基
<i>c</i>	<i>f</i> :3
<i>a</i>	<i>fc</i> :3
<i>b</i>	<i>fca</i> :1, <i>f</i> :1, <i>c</i> :1
<i>m</i>	<i>fca</i> :2, <i>fcab</i> :1
<i>p</i>	<i>fcam</i> :2, <i>cb</i> :1

FP-树 结构的优点

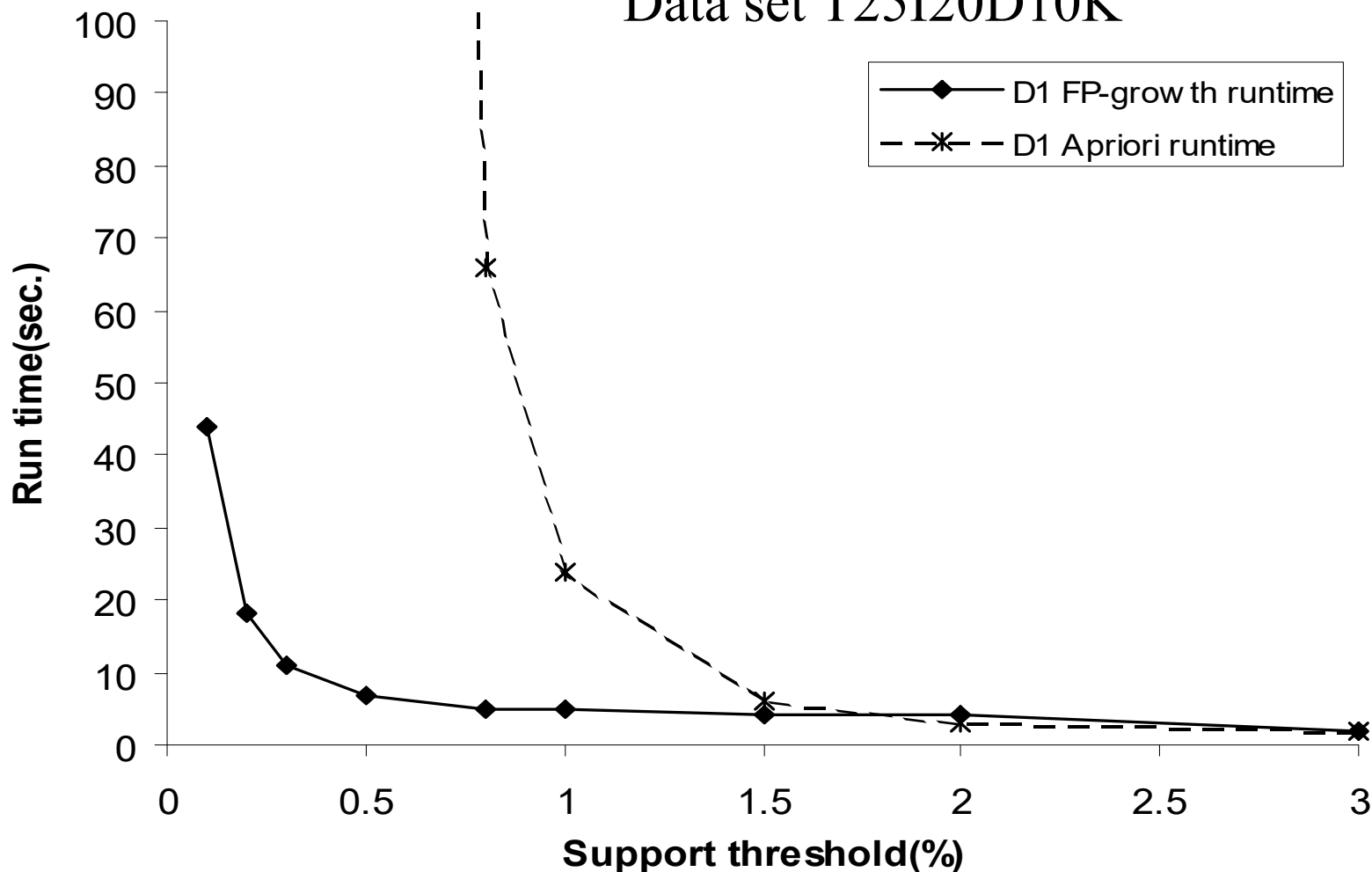
- 完整性
 - 保持了频繁项集挖掘的完整信息
- 紧密性
 - 减少不相关的信息，去掉了非频繁的项
 - 项按支持度降序排列：越频繁出现，越可能被共享
 - 不会比原数据库更大 (不计结点链接和计数域)

4.3 FP-growth算法



FP-growth 与 Apriori的比较

Data set T25I20D10K

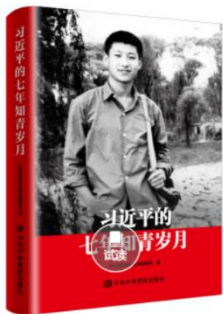


- 4.1 概述
- 4.2 Apriori 算法
- 4.3 FP-growth 算法
- 4.4 推荐算法 
- 4.5 小结

4.4 推荐算法



图书 > 政治/军事 > 中国政治 > 中共中央党校出版社 > 习近平的七年知青岁月



习近平的七年知青岁月

单品团购100册以上可致电4006186622
中央党校采访实录编辑室 著

京东价 **¥49.40** [6.5折] [定价:¥76.00] 降价通知

增值业务 助力环保,传递知识,旧书换新

配 送 至 天津南开区广开街道 有货

京东物流 京准达 | 211限时达 | 预约送货

由 京东 发货,并提供售后服务,23:00前下单,预计明天(09月28日)送达

重 量 0.775kg

服务支持 **放心购** 上门换新 | 破损包退换 | 闪电退款
可配送全球49元免基础运费

- 选择系列
- 习近平四省足迹系列
 - 习近平强军思想学习问答 (32开)
 - 习近平生态文明思想学习纲要32开
 - 习近平强军思想学习问答
 - 习近平福建足迹
 - 习近平浙江足迹
 - 习近平正定足迹
 - 习近平生态文明思想学习纲要16开
 - 习近平上海足迹

企业批量购书

人气单品

七日畅销榜

新书热卖榜



中国共产党的九十年
(套装共3册) (2016年)

¥126.60



中华人民共和国民法典
(实用版批注咨询京东)

¥26.00



干在实处 走在前列:推
进浙江新发展的思考与

¥44.80



课程思政:从理念到实
践

¥46.70



中华人民共和国民法典
(大字版32开大字条旨)

¥8.90

本店好评商品



思想的力量:新一届中央领导集体治
国理政的基本思路

¥29.00

关注



中国道路:走向现代化的全新选择
(精)

¥51.00

关注



直通两会2019:两会热点面对面 (视
频书)

¥39.80

关注



中国政治的逻辑:理论与实践

¥54.90

关注

猜你喜欢



¥107.80



¥58.80



¥87.80



¥91.00



¥156.40



¥51.70

4.4 推荐算法



电商

☆ 与您浏览过的商品相关的推荐



与您浏览过的商品相关的推荐

浏览此商品的顾客也同时浏览

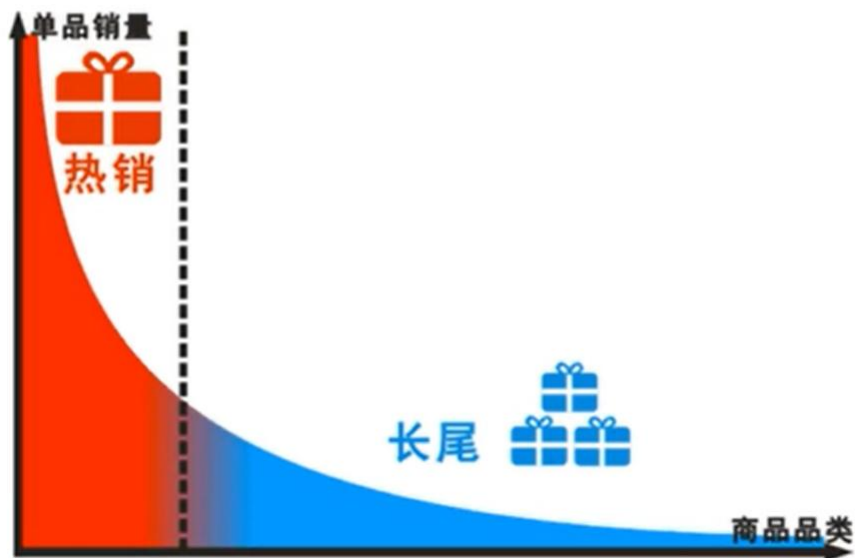
浏览此商品的顾客也同时浏览



Why：为什么要做推荐？



长尾挖掘



□ 实例

- 商品、服务、广告

□ 原因

- 高斯分布 VS 长尾分布

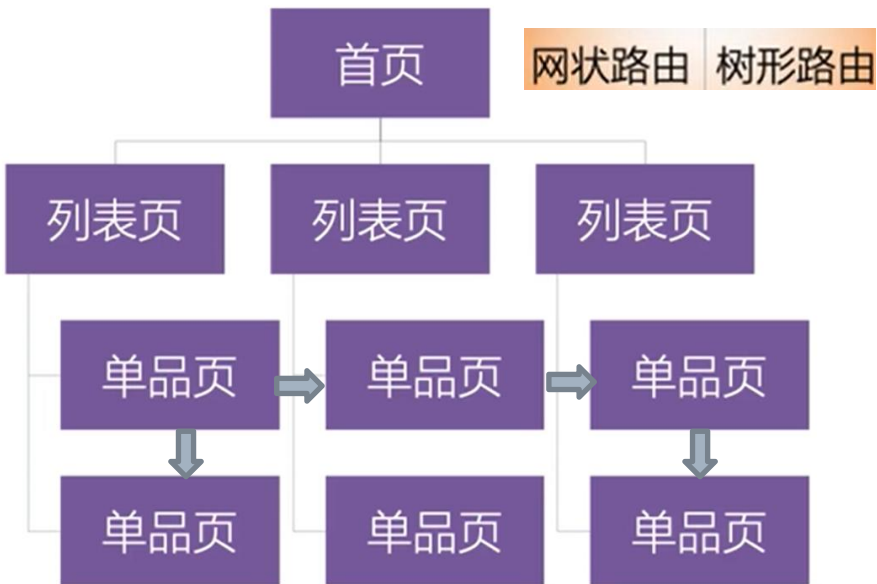
□ 问题

- 如何找到匹配的长尾商品

□ 价值

- 用户：找到自己想要的商品
- 平台：提升长尾商品价值

4.4 推荐算法



宝贵的	• 外部成本 • 内部成本
不确定的	• 用户意图
差异化的	• 整体用户群 • 到达具体页面/位置的用户
稀疏的	• 差异化的另一面 • 每一“种”用户的数量

Screenshot of a book recommendation interface from a Chinese e-commerce platform.

人气单品 | 七日畅销榜 | 新书热卖榜

Book Title	Price
MongoDB权威指南 第3版 (图灵出品)	¥102.50
创想大数据——企业级数据仓库项目实施 (电商)	¥167.80
精益数据分析 (图灵出品)	¥60.20
SQL必知必会 第5版 (图灵出品)	¥32.90
SQL从入门到精通 (微课视频版) sql基础教程进阶	¥79.80
白话统计 (博文视点出品)	¥68.80

商品介绍 | 规格与包装 | 售后保障 | 商品评价(500+) | 本店好评商品 | [加入购物车](#)

出版社: 人民邮电出版社 | ISBN: 9787115241009 | 版次: 2 | 商品编码: 12918545
品牌: ituring | 包装: 平装 | 丛书名: 图灵计算机科学丛书 | 开本: 16开
出版时间: 2020-12-01 | 用纸: 胶版纸 | 页数: 463 | 正文语种: 中文

编辑推荐

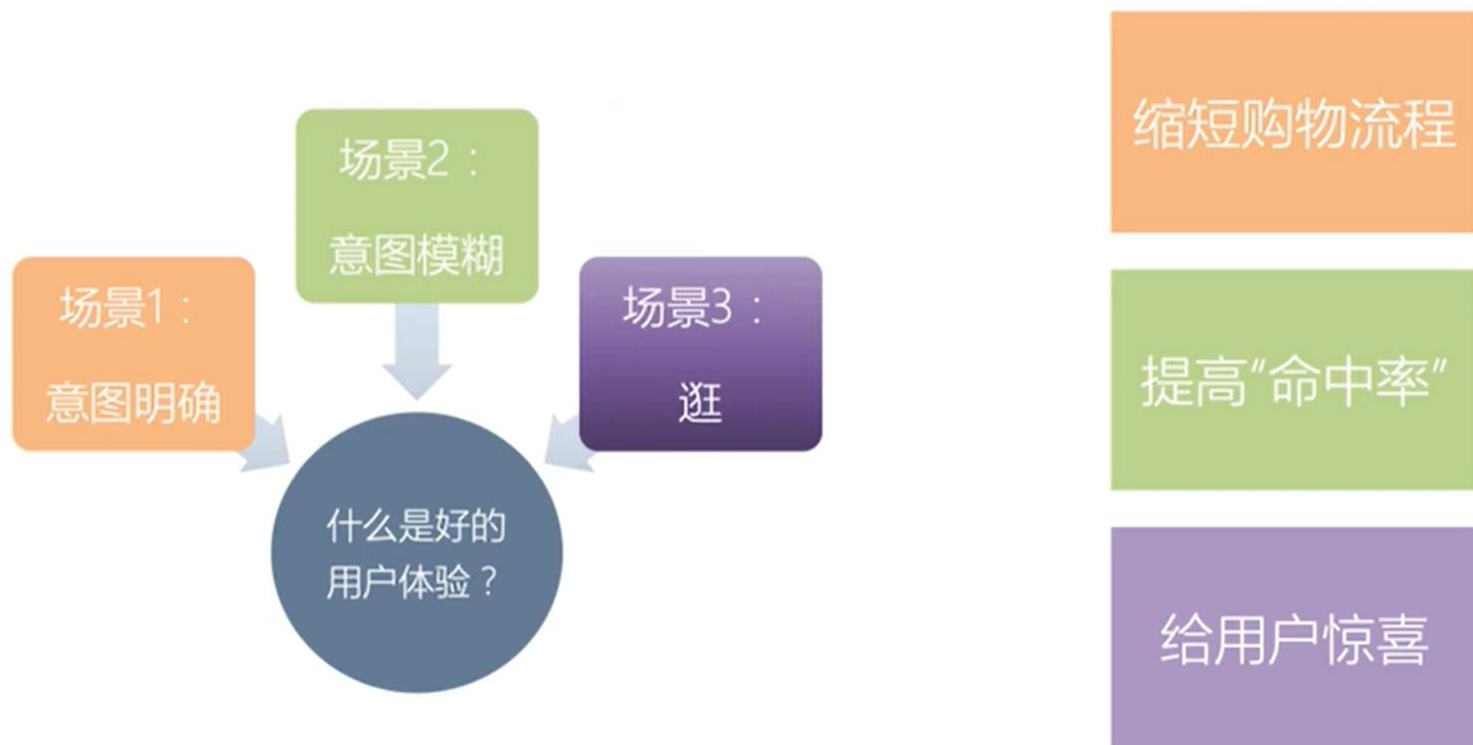
《数据挖掘导论(完整版)》介绍了数据挖掘,涵盖了五个主题:数据、分类、关联分析、聚类和异常检测。除异常检测外,每个主题具有两章。前一章涵盖基本概念、代表性算法和评估技术,而后一章讨论概念和算法。这样读者在透彻地理解数据挖掘的基础的同时,还能够了解更多重要的主题。《数据挖掘导论(完整版)》是明尼苏达大学和密歇根州立大学数据挖掘课程的教材,由于独具特色,正式出版之前已经被斯坦福大学、得克萨斯大学奥斯汀分校等众多名校采用。《数据挖掘导论(完整版)》特色与许多其他同类图书不同,《数据挖掘导论(完整版)》将重点放在如何用数据挖掘知识解决各种实际问题。

编辑推荐

内容简介
作者简介
目录



用户体验

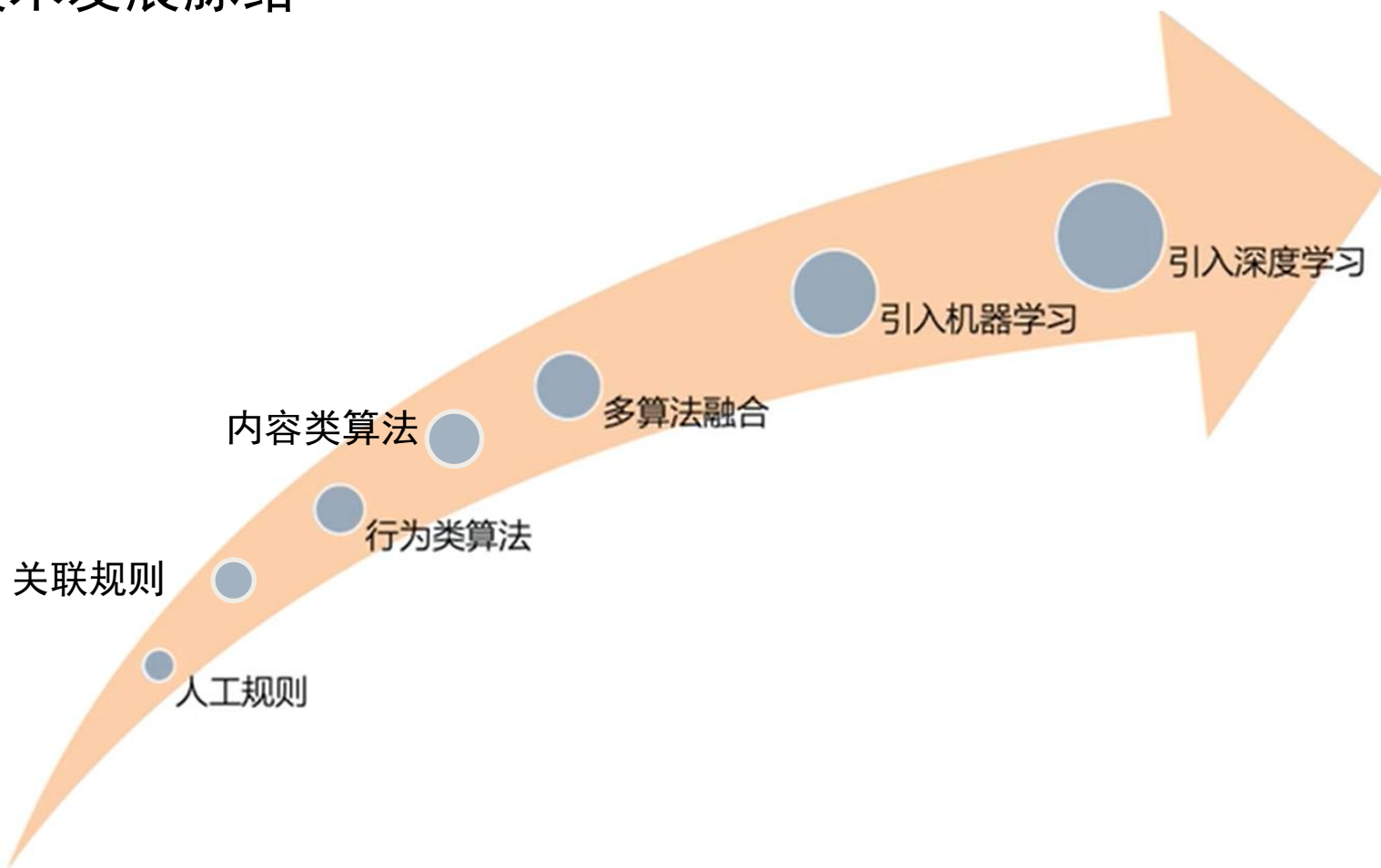


4.4 推荐算法



		1	2	3	4	5	6
							
A		4	3			5	
B		5		4		4	
C		4		5	3	4	
D			3				5
E			4				4
F				2	4		5

技术发展脉络



4.4.1 基于流行度的推荐算法



- 比较简单粗暴，主要是对热点商品或者信息的推荐。例如利用分享率、点击量、得票数等数据来按某种热度排序来推荐给用户。
- 优点：简单，适用于刚注册的新用户，能够解决对新用户进行推荐的冷启动问题。
- 缺点：分数是人为设计的，并且无法针对用户提供个性化的推荐。算法可做一些优化，比如加入用户分群的流行度排序，例如把热榜上的体育内容优先推荐给体育迷，把政要热文推给热爱谈论政治的用户。

Y Hacker News new past comments ask show jobs submit		
1. ▲ Ghidra, NSA's reverse-engineering tool (nsa.gov)	222 points by twodayslate 2 hours ago hide 105 comments	
2. ▲ The Troika Laundromat (occrp.org)	85 points by jumelles 2 hours ago hide 11 comments	
3. ▲ Graphene-Based Filter Purifies Salt Water for Drinking (bbc.com)	45 points by 11thEarlOfMar 2 hours ago hide 15 comments	
4. ▲ Oxide: A Formal Semantics for Rust (arxiv.org)	86 points by rachitnigam 3 hours ago hide 4 comments	
5. ▲ Achieving 100k connections per second with Elixir (stressgrid.com)	403 points by slashdotdash 9 hours ago hide 86 comments	
6. ▲ Common Lisp Koans (github.com)	97 points by bibyte 5 hours ago hide 11 comments	
7. ▲ A surprisingly arcane little Unix shell pipeline example (utoronto.ca)	39 points by zdw 3 hours ago hide 3 comments	
8. ▲ chunkwm — a tiling window manager for macOS (koelshliya.github.io)	90 points by firloop 3 hours ago hide 39 comments	
9. ▲ Mammalian Near-IR Vision Through Injectable, Self-Powered Retinal Nanoantennae (cell.com)	75 points by mhb 5 hours ago hide 35 comments	
10. ▲ FaunaDB 2.5.4 (jepsen.io)	228 points by aphyrr 11 hours ago hide 63 comments	

- $$\text{score} = \frac{P-1}{(T+2)^G}$$
- P：帖子得票数
- T：帖子发布时间
- G：超参数，控制衰减速度

- 首要目标是挖掘出关联规则，同时被很多用户购买的物品集合，这些集合内的物品可以相互进行推荐。目前关联规则挖掘算法主要从Apriori和FP-Growth两个算法发展演变而来。基于关联规则的推荐系统一般转化率较高，因为当用户已经购买了频繁集合中的若干项目后，购买该频繁集合中其他项目的可能性更高。
- 缺点：
 - 计算量较大，但是可以离线计算，因此影响不大。
 - 由于采用用户数据，不可避免的存在冷启动和稀疏性问题。
 - 存在热门项目容易被过度推荐的问题。

4.4.3 基于协同过滤的推荐算法

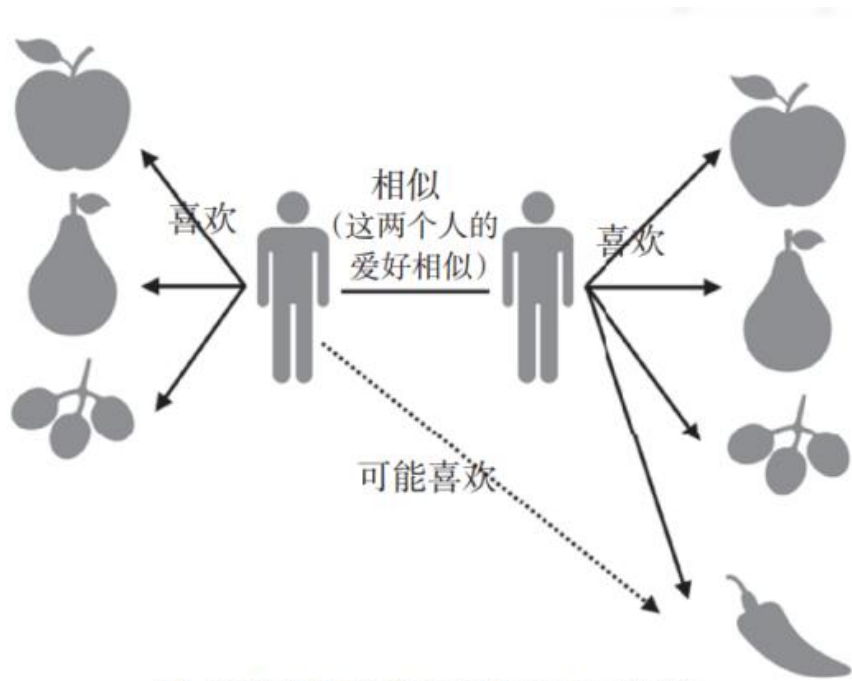


- 协同过滤是一种广泛采用的推荐方法。基于“物以类聚，人以群分”的假设，喜欢相同物品的用户更有可能具有相同的兴趣。
- 一般应用于有用户评分的系统之中，通过分数去刻画用户对于物品的喜好。协同过滤被视为利用集体智慧的典范，不需要对项目进行特殊处理，而是通过用户建立物品与物品之间的联系。
- 主要有两种类型：基于用户(User-based)的推荐和基于物品(Item-based)的推荐。

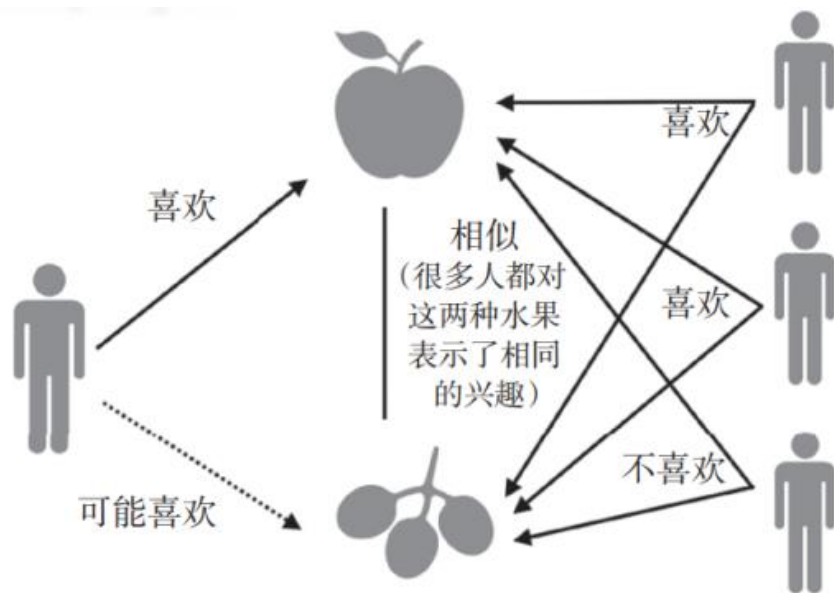
4.4.3 基于协同过滤的推荐算法



- 基于用户的协同过滤算法：“人以群分”，找到与用户兴趣相同的用户（有过类似的行为），将兴趣相同的用户浏览过物品推荐给用户。
- 基于物品的协同过滤推荐算法：“物以类聚”，如果有很多用户都对某两个物品有相似的偏好，说明这两个物品是“相似”的，通过推荐与用户喜欢过的相似的物品这种方式为用户提供个性化推荐。



“人以群分”的基于用户的协同过滤



“物以类聚”的基于物品的协同过滤

4.4.3.1 基于用户的推荐算法



1 2 3 4 5 6



4	3			5	
5		4		4	
4		5	3	4	
	3				5
	4				4
		2	4		5

计算每两个用户间的相似度

4.4.3.1 基于用户的推荐算法



- 根据所有用户对物品或者信息偏好（评分），发现与当前用户口味和偏好相似的“邻居”用户群，在一般应用中是采用计算K近邻的算法；基于这 K个邻居的历史偏好信息，为当前用户进行推荐。
- 优点在于推荐物品之间在内容上可能完全不相关，因此可以发现用户的潜在兴趣，并且针对每个用户生成其个性化的推荐结果。
- 缺点在于一般的Web系统中，用户的增长速度都远远大于物品增长速度，因此其计算量的增长巨大，系统性能容易成为瓶颈。因此在业界中单纯的使用基于用户的协同过滤系统较少。

4.4.3.1 基于用户的推荐算法



□ 基于用户(User-based)的推荐

- 将用户向量化表示

- $u:[1,1,0,1,...]$, 每个维度表示一个物品

□ 使用向量空间模型(VSM)计算基础相关性

$$\text{sim}(u_i, u_j) = \cos(\theta_{\{u_i, u_j\}}) = \frac{u_i \cdot u_j}{|u_i| |u_j|}$$

□ 利用基础相关性+用户原始行为计算最终推荐结果

$$\text{rel}(\text{user}_i, \text{item}_j) = \sum_k \text{sim}(\text{user}_i, \text{user}_k) \text{rate}(\text{user}_k, \text{item}_j)$$

4.4.3.1 基于用户的推荐算法



- 是否需要计算每两个用户之间的相似度?
 - 不需要，大量用户间相关性可预判为0
 - $[1,2,0] * [0,0,3]$ ，结果必然为0
 - 通过倒排表实现高效筛选
- 是否需要使用一个用户的全部相关用户来计算?
 - 不需要，相关性影响衰减非常快，只需取TopK就可
 - 在哪里去除?用户相关性计算阶段
- 没有rate怎么办?
 - implicit feedback :点击、浏览、购买.....

4.4.3.2 基于物品的推荐算法



- 使用所有用户对物品或者信息的偏好（评分），发现物品和物品之间的相似度，根据用户的历史偏好信息，将类似的物品推荐给用户。
- 可以看作是关联规则推荐的一种退化，但由于协同过滤更多考虑了用户的实际评分，并且只是计算相似度而非寻找频繁集，可以认为基于物品的协同过滤准确率较高并且覆盖率更高。
- 同基于用户的推荐相比，基于物品的推荐应用更为广泛，扩展性和算法性能更好。由于项目的增长速度一般较为平缓，因此性能变化不大。缺点就是无法提供个性化的推荐结果。

4.4.3.2 基于物品的推荐算法



		1	2	3	4	5	6
							
A		4	3			5	
B		5		4		4	
C		4		5	3	4	
D			3				5
E			4				4
F				2	4		5



相似

相似度0.27



不相似

相似度0

□ 基于物品(Item-based)的推荐

- 将物品向量化表示

- $item: [1, 1, 0, 1, \dots]$, 每个维度表示一个用户

□ 使用向量空间模型(VSM)计算基础相关性

$$sim(item_i, item_j) = \cos(\theta_{\{item_i, item_j\}}) = \frac{item_i \cdot item_j}{|item_i| |item_j|}$$

□ 利用基础相关性+用户原始行为计算最终推荐结果

$$rel(user_i, item_j) = \sum_k sim(item_k, item_j) rate(user_i, item_k)$$

4.4.3.2 基于物品的推荐算法



		1	2	3	4	5	6
							
A		4	3			5	
B		5		4		4	
C		4		5	3	4	
D			3				5
E			4				4
F				2	4		5

✓ 相似
相似度0.27

✗ 不相似
相似度0

		1	2	3	4	5	6
							
1			0.27	0.79	0.32	0.99	0
2		0.27					
3		0.79					
4		0.32					
5		0.99					
6		0					

4.4.3 基于协同过滤的推荐算法



基于用户	基于物品
结果群体化、扩展化	结果个性化、精准化
适用于用户量相对小于物品量的场景，例如新闻推荐系统	适用于物品量相对小于用户量的场景，例如电商平台
对用户新行为敏感度低	对用户新行为敏感度高
对物品新行为敏感度高	对物品新行为敏感度低
推荐理由说服力较弱	推荐理由说服力较强

□ 优点：

- 不需要对物品或者用户进行严格的建模，而且不要求物品的描述是机器可以理解的，所以这种方法也是领域无关的。
- 计算出来的推荐是开放的，可以共用他人的经验，很好地支持用户发现潜在的兴趣偏好。

□ 缺点：

- 核心是基于历史数据，对新物品和新用户都有“冷启动”的问题。
- 推荐的效果依赖于用户历史偏好数据的多少和准确性。
- 在大部分的实现中，用户历史偏好是用稀疏矩阵进行存储的，而稀疏矩阵上的计算有些明显的问题，包括可能少部分人的错误偏好会对推荐的准确度有很大的影响等等。
- 对于一些特殊品味的用户不能给予很好的推荐。
- 由于以历史数据为基础，抓取和建模用户的偏好后，很难利用获取的用户偏好演变，从而导致这个方法不够灵活。

- 多用于一些资讯类的应用上，针对文章（电影音乐）本身抽取一些tag作为其关键词，继而可以通过这些tag来评价两篇文章的相似度。
- 优点：易于实现，不需要用户数据因此不存在稀疏性和冷启动问题。基于物品本身特征推荐，因此不存在过度推荐热门的问题。
- 缺点：
 - 抽取的特征既要保证准确性又要具有一定的实际意义，否则很难保证推荐结果的相关性。豆瓣网采用人工维护tag的策略，依靠用户去维护内容的tag的准确性。
 - 推荐的Item可能会重复，典型的就新闻推荐，如果你看了一则津云上的新闻，很可能推荐的新闻和你浏览过的，内容一致。

4.4.4 基于内容的推荐算法



Term Frequency (TF)

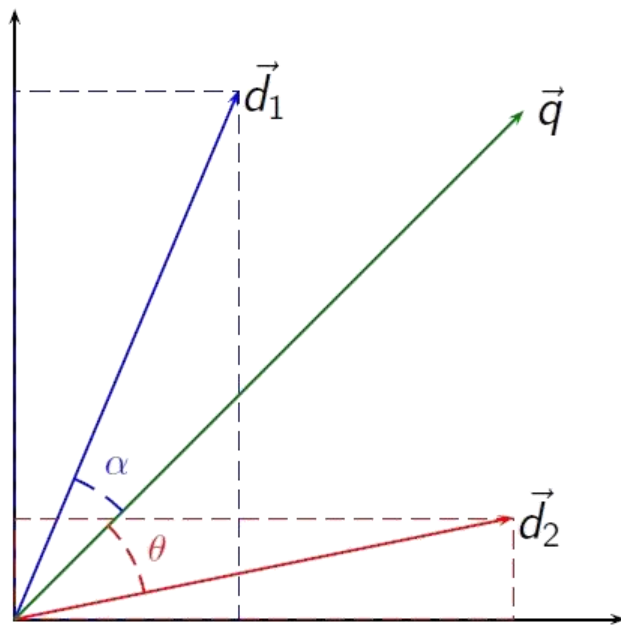
$$tf(t, d) = \frac{n_{t,d}}{\sum_k n_{k,d}}$$

Inverse Document Frequency (IDF)

$$idf(t, D) = \log \frac{|D|}{|\{d \in D: t \in d\}|}$$

TF-IDF

$$tf(t, d) * idf(t, D)$$



- 把文本表示成一个向量
- 余弦相似度Cosine Similarity

$$sim(p, q) = \cos(\theta) = \frac{p \cdot q}{|p| \cdot |q|}$$

■ 前述方法的缺点

- 启发式，无明确目标
- 相对静态，反应慢
- 考虑维度较少，个性化程度不足
- 纯手工，改进空间小

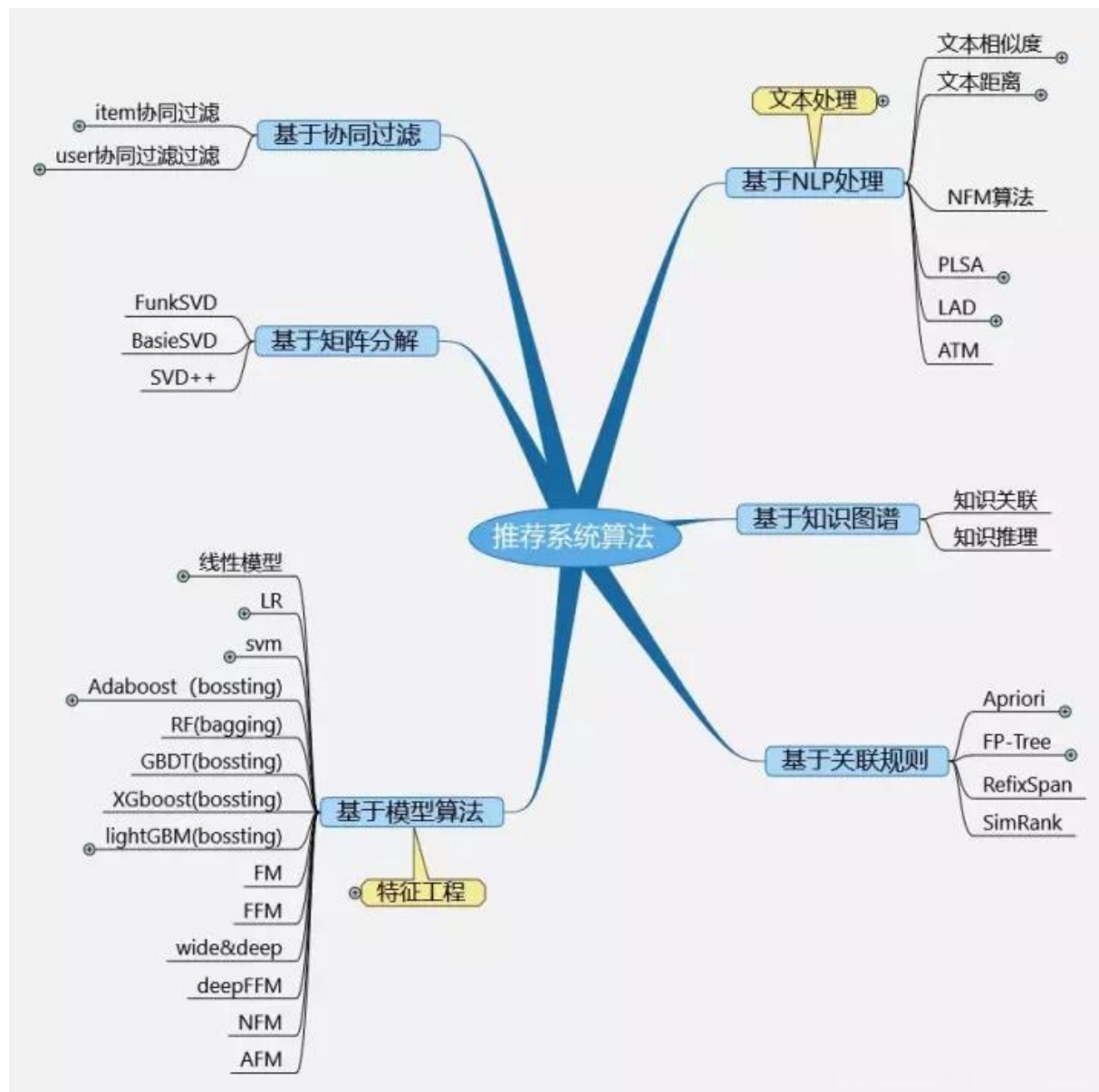
■ 机器学习的优点

- 对齐明确目标，成熟的优化方法
- 时效性灵活，反映可快可慢
- 特征维度丰富，个性化程度细腻
- 理论储备丰富，发展空间大

基于模型的方法有很多，主要是使用常用的机器学习算法对目标用户建立推荐算法模型，然后对用户的爱好进行预测推荐以及对推荐的结果打分排序等。

- 常用的浅层模型包括LR, GBDT、特征组合等方法
- 半深度模型包括FM、FFM
- 深度模型包括Wide & Deep、embedding、LSTM、ESMM、TDM等

4.4.5 基于模型的推荐算法



nature machine intelligence

Article

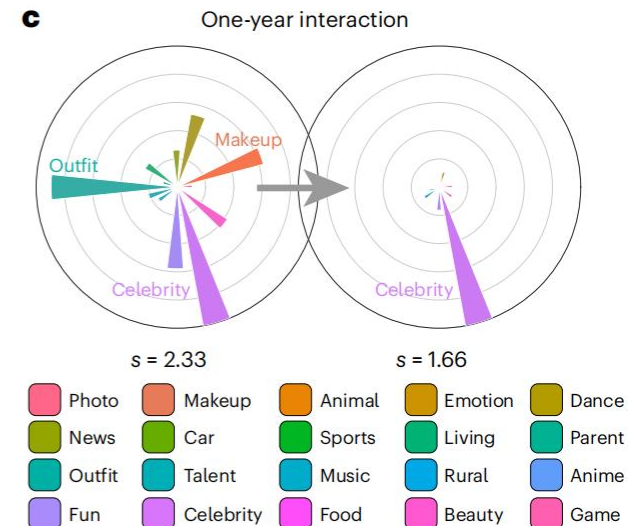
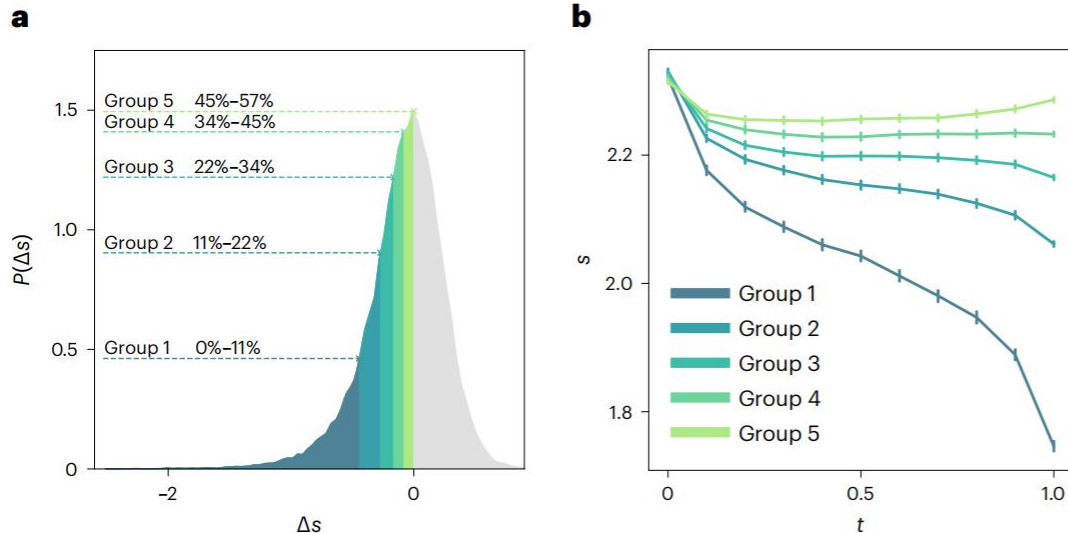
<https://doi.org/10.1038/s42256-023-00731-4>

Human–AI adaptive dynamics drives the emergence of information cocoons

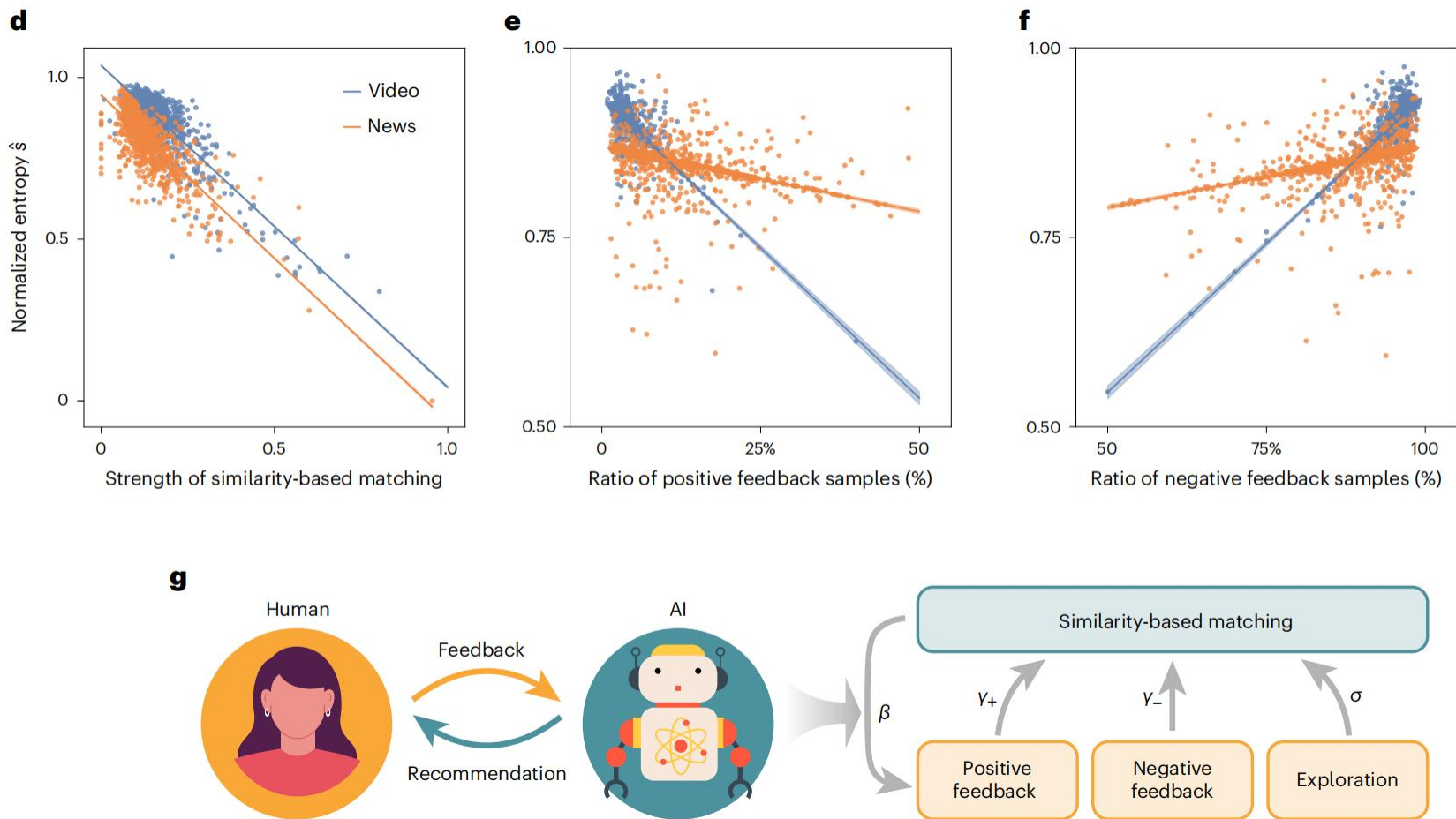
Received: 30 March 2023

Jinghua Piao^{1,3}, Jiazhen Liu^{1,3}, Fang Zhang², Jun Su² & Yong Li¹✉

Accepted: 6 September 2023



4.4.5 基于模型的推荐算法



动态的人机交互过程

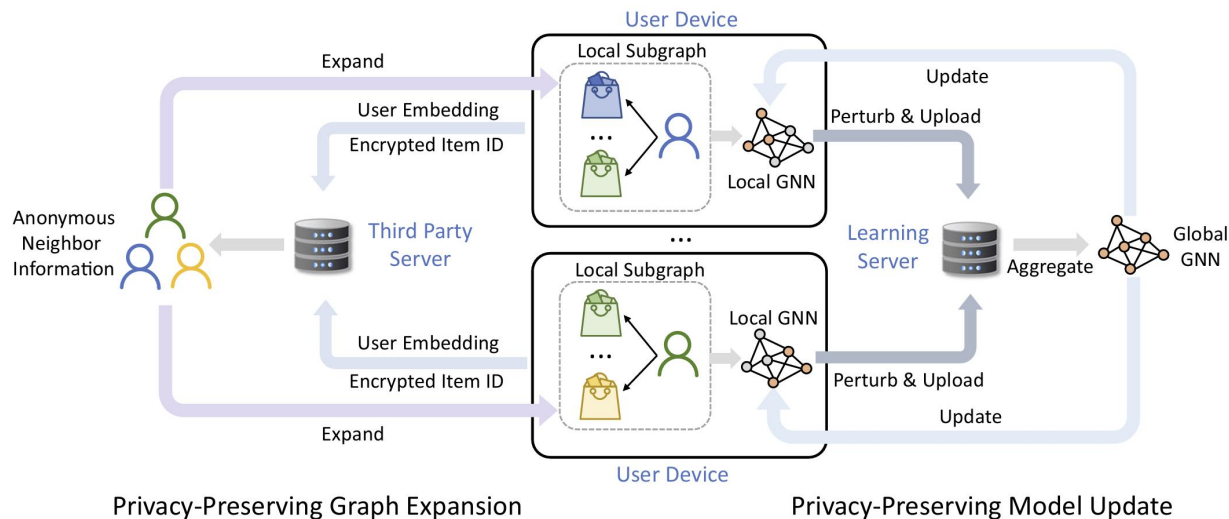
随机游走/图排序	PersonalRank, ItemRank, SimRank, RWR	图上的概率传播
图嵌入	DeepWalk, node2vec, LINE, metapath2vec	节点低维向量化
同构 GNN	GCN, GraphSAGE, GAT, PinSage, NGCF, LightGCN	邻域消息聚合
异构图 GNN	HetGNN, HAN, MAGNN, RGCN, HGT	元路径/关系聚合
知识图谱 GNN	RippleNet, MKR, KGCN, KGAT, KGIN	知识图谱增强
社交图 GNN	GraphRec, DiffNet, SocialGCN, MHCN	社交影响传播
会话图 GNN	SR-GNN, GGNN, TAGNN, LESSR	会话序列图建模
图对比学习	SGL, SimGCL, XSimGCL, NCL	自监督对比
图 Transformer/LLM	GraphGPT, LLaGA, GPT4Rec	大模型+图
联邦图推荐	FedPerGNN, FedGraphNN	隐私保护协同训练

从浅层图嵌入走向深度 GNN，从同构二部图走向异构图/知识图谱，从集中式走向联邦与隐私保护，从监督学习走向自监督与对比学习，并开始与大语言模型融合。

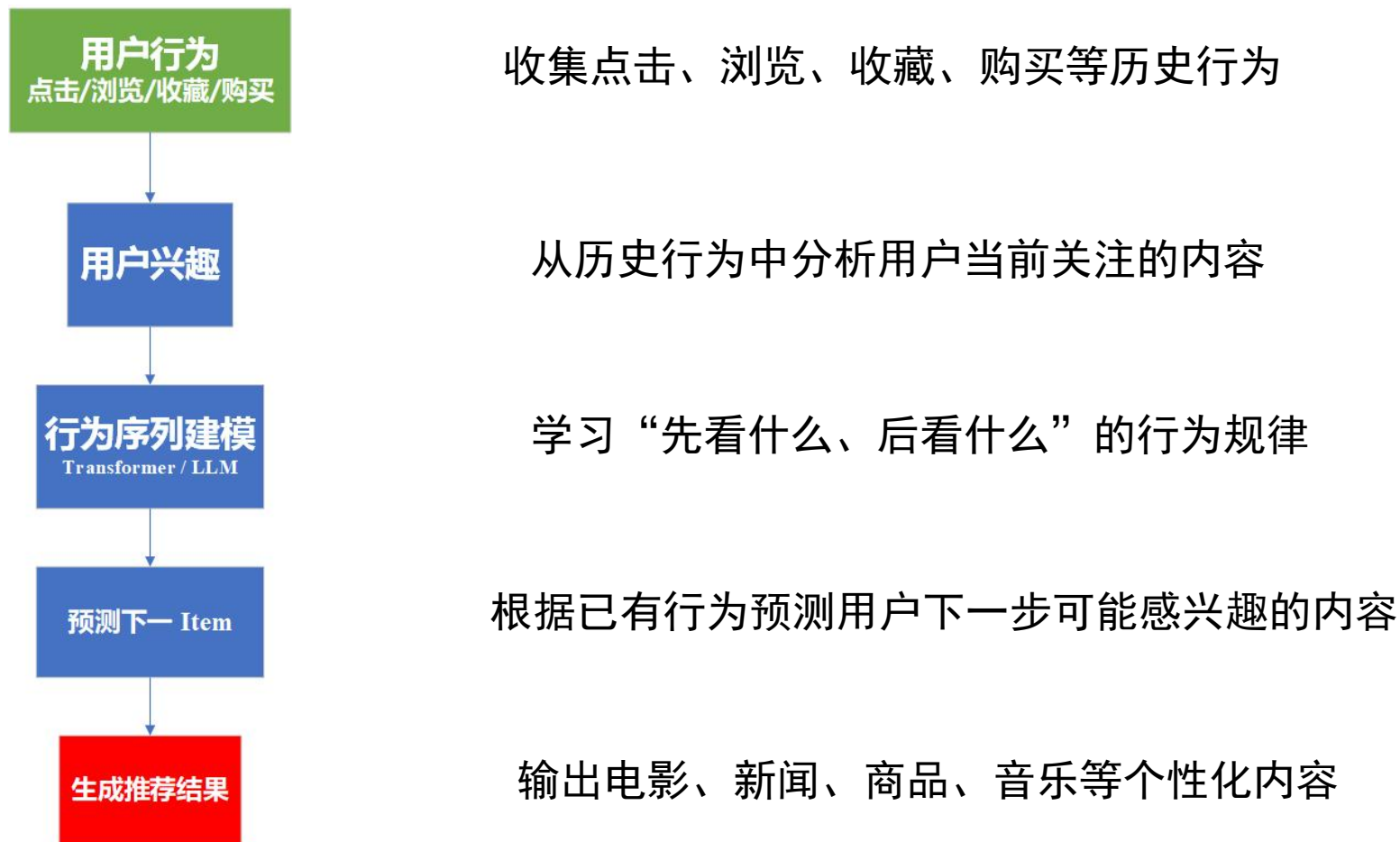
nature communications

[View](#)[Explore content](#) ▾ [About the journal](#) ▾ [Publish with us](#) ▾[nature](#) > [nature communications](#) > [articles](#) > articleArticle | [Open access](#) | Published: 02 June 2022

A federated graph neural network framework for privacy-preserving personalization

[Chuhan Wu](#), [Fangzhao Wu](#) , [Lingjuan Lyu](#), [Tao Qi](#), [Yongfeng Huang](#)  & [Xing Xie](#)[Nature Communications](#) **13**, Article number: 3091 (2022) | [Cite this article](#)

- 多用于大规模、动态兴趣和复杂用户行为的推荐场景，通过 Transformer、生成模型或大语言模型对用户历史行为、物品信息和上下文进行统一建模，并直接生成用户可能感兴趣的 Item 或 Item Token，而不是传统的候选集评分和排序。



范式	核心机制	代表性方法	核心优势	主要挑战
判别式	LLM作为特征编码器	LLMEmb, CEILLM	实现简单，增强现有模型	未充分利用LLM推理能力
生成式	推荐即文本生成	P5, TALLRec, RecLM-cgen	统一框架，可解释性强	幻觉，推理延迟
RAG	检索+生成	CFRAG	引入外部知识，缓解幻觉	检索质量依赖
图/多模态	LLM+结构化/多模态信息	KGGLM, Molar	融合知识图谱与视觉信号	模态对齐复杂
智能体	LLM作为Agent大脑	自主工具学习Agent	多轮交互，主动规划	工程复杂，延迟高

当前趋势是混合与融合：判别式方法为生成式提供高质量的语义表示，RAG为生成式注入实时知识，图结构为LLM补充结构化推理能力。

■ 优点：

- 能够直接建模用户行为序列，捕捉用户兴趣随时间的变化；
- 可以利用文本、标签等丰富的语义信息；
- 推荐过程从传统的候选集排序转变为生成推荐结果，能够减少对人工设计特征和候选集的依赖。

■ 缺点：

- 模型结构复杂，训练和推理成本较高；
- 对于推荐结果的准确性和可控性仍然需要进一步约束。

■ 典型方法：Transformer-based Sequential Recommendation、Generative Retrieval、LLM-based Recommendation。

- 4.1 概述
- 4.2 Apriori 算法
- 4.3 FP-growth 算法
- 4.4 推荐算法
- 4.5 小结 

□ 经典算法

- Apriori
- FP-growth算法

□ 推荐算法

- 基于协同过滤的算法
- 基于模型的算法
- 基于图的算法
- 基于大模型的算法

THE END

